What-Happens-Next Modal: Track A & Track B Technical Report

Something-Something V2 action anticipation (Kaggle)
Track A: closed-world (from scratch) Track B: open-world (pretrained allowed)

May 17, 2026

Abstract

This report documents the engineering work on the *What-Happens-Next* Kaggle competition (Something-Something V2 action anticipation) for **Track A** (closed world: no external pretraining) and **Track B** (open world: pretrained backbones allowed).

Track A is organised around four progressive improvements on a shared supervised pipeline. **Training pipeline foundations** corrects validation accounting, initialization, AMP, optimizer/scheduler resume state, and untrained-class masking at submit time. **Architectural and training enhancements** add DropPath, an attention-pool head, EMA checkpointing, and flip-TTA with automatic left/right class-pair remap. **Self-supervised pretraining** covers V-JEPA-style clip SSL for the TSM ResNet path and the overnight VideoMAE fish-matrix sweep documented in `SSL.md`. **Data augmentation, class balance, and TTA improvements** add class-balanced sampling/loss, motion-aware time-reversal augmentation, multi-scale TTA, and logit adjustment. Empirical runs (dual-stream CNN, supervised ViT ablations, and SSL slots) are recorded in the Track A run logbook.

Track B focuses on a frozen V-JEPA 2 backbone with a trainable probe, including zero-shot evaluation, heavy augmentation, and large-model overnight runs. All pipeline changes default to off unless enabled in Hydra.

Contents

1 Context

1.1 Competition rules

The Kaggle competition *Can a model predict the future?* provides short video clips from the Something-Something V2 dataset and asks participants to predict the action that will occur next, given only the beginning of each clip. Track A (*Closed World*) is the focus of this work: **models must be trained from scratch using only the provided data**; no external dataset and no pretrained weights are allowed. Submissions are evaluated by Top-1 classification accuracy (with Top-5 as a secondary metric) on a held-out test set whose public/private split is hidden from participants.

1.2 Starting point and observed problems

Before the work in this report, the supervised Track-A pipeline was a TSM-ResNet50 trained with Adam, label smoothing, RandAugment-T temporal augmentations, FrameMixUp, an 80/20 internal split of the training data for validation, and a cosine learning-rate schedule. Its best public Kaggle scores were:

Submission	Internal val Top-1	Public Top-1
Initial run (epoch 29)	0.4582	0.3207
Resumed run (epoch 60)	0.5666	0.3584

The roughly 21 percentage-point gap between internal validation and the public leaderboard pointed at a collection of issues, the most important of which were identified in a project review:

1. **Leaky validation split**. The 80/20 split was carved out of `data/train/` rather than using the official held-out `data/val/` folder. Validation samples were therefore drawn from the same recording sessions and demographics as training, producing optimistically biased numbers.
2. **Suboptimal initialisation**. The ResNet-50 backbone was initialised with Xavier weights, which is well-suited to networks with sigmoid/tanh activations but is documented to under-perform Kaiming He initialisation on deep ReLU networks trained from scratch.
3. **No mixed-precision training**. Forward and backward passes ran in full FP32, leaving $\approx 1.5{-}2\times$ of GPU throughput on the table on Ampere or newer hardware.
4. **Optimizer / scheduler / scaler state lost on resume**. Only the model weights and the epoch counter were checkpointed, so restarting a long run forced the optimizer (Adam moments) and the cosine LR schedule to restart from scratch as well.
5. **Missing class 27**. The "50 classes" described in the competition page is in practice 33 unique class indices on disk with class 27 entirely absent. The submission pipeline did nothing to prevent the network from emitting this never-trained label, meaning $\approx 3\%$ of test predictions could land on it.
6. **No EMA, no TTA, no SSL pretraining.** The pipeline used none of the well-known accuracy-boosting techniques that are standard for video classification on small datasets.

The remainder of this document describes the changes that addressed each of these issues, in the order they were introduced.

2 Training Pipeline Foundations

Phase 1 comprises five low-risk, high-return changes. All of them were introduced behind explicit configuration switches so that historical runs can still be reproduced bit-for-bit by setting the new switches to their "off" defaults.

2.1 Official validation split

Logic. A train/val split that overlaps with the test distribution understates the generalisation gap. Switching to the official `data/val/` folder makes the validation score a faithful proxy for the public leaderboard score and therefore drives early stopping, model selection and hyperparameter choices in the right direction.

Implementation. A new boolean `dataset.use_official_val` (default `false`) was added to `configs/data/default.yaml`. When set to `true`, the trainer bypasses the internal 80/20 split and instead enumerates every video under `data/val/`.

```
use_official_val = bool(cfg.dataset.get("use_official_val", False))
if use_official_val:
    val_dir_for_val = Path(cfg.dataset.val_dir).resolve()
    val_samples = collect_video_samples(val_dir_for_val)
    train_samples = all_samples
    print(f"[data] use_official_val=true: train={len(train_samples)}, "
          f"val={len(val_samples)} (from {val_dir_for_val})")
else:
    train_samples, val_samples = split_train_val(
        all_samples, val_ratio=float(cfg.dataset.val_ratio),
        seed=int(cfg.dataset.seed))
```

2.2 Kaiming He initialisation

Logic. For a Conv→BN→ReLU stack trained from scratch, the variance of pre-activations grows or shrinks geometrically with depth unless the initial Conv weights are scaled with the Kaiming He recipe [1, Sec. 2.2]:

$$\mathrm{Var}(W_{i,j}) = \frac{2}{\text{fan\_out}}.$$

Xavier initialisation, which uses $2/(\text{fan\_in} + \text{fan\_out})$, under-shoots this variance by roughly $2\times$ per layer for ReLU networks and is empirically a poor choice for from-scratch ResNet training.

Implementation. The `_init_weights_xavier` method of `AvancedResNet50TSM` was renamed to `_init_weights` and rewritten to walk every submodule:

```
def _init_weights(self) -> None:
    for module in self.modules():
        if isinstance(module, nn.Conv2d):
            nn.init.kaiming_normal_(module.weight,
                mode="fan_out", nonlinearity="relu")
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.BatchNorm2d):
            if module.weight is not None:
                nn.init.ones_(module.weight)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
    nn.init.normal_(self.classifier.weight, mean=0.0, std=0.01)
    if self.classifier.bias is not None:
        nn.init.zeros_(self.classifier.bias)
```

The classifier head is initialised with a small-Gaussian ($\mathcal{N}(0, 0.01^2)$) following the original ResNet and TSM recipes; BN affine parameters are set to $\gamma = 1, \beta = 0$.

2.3 Automatic Mixed Precision (AMP)

Logic. On Ampere, Hopper and newer GPUs, running the forward pass in FP16 (or BF16) and only the master copy of the optimizer state in FP32 yields 1.5–2× throughput at no measurable accuracy cost when paired with PyTorch's gradient scaler.

Implementation. A new boolean `training.amp` (default `false`) gates a `torch.amp.GradScaler` that is created in `train.py` and threaded through the engine helpers. `train_one_epoch` and `evaluate_epoch` both received two new keyword arguments, `scaler` and `amp_dtype`, and wrap their forward and backward passes in `torch.amp.autocast(device_type="cuda", dtype=amp_dtype)`. When `scaler` is `None` (the default), the code path is byte-for-byte identical to the legacy full-precision implementation.

```
amp_enabled = bool(cfg.training.get("amp", False)) \
            and device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=True) \
        if amp_enabled else None

# ...inside train_one_epoch:
with torch.amp.autocast(device_type="cuda",
                        enabled=use_amp,
                        dtype=amp_dtype):
    logits = model(videos)
    loss = loss_fn(logits, target)
if use_amp:
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss.backward()
    optimizer.step()
```

2.4 Persistent optimizer / scheduler / scaler state

Logic. A long training run inevitably gets restarted (scheduled maintenance, a transient OOM, an accidental `Ctrl-C`). If only the model weights are persisted, the optimizer's running moments, the cosine LR schedule and the GradScaler all reset, which causes a visible accuracy dip for several epochs after each restart and makes "resume from epoch k" a much weaker recovery than "never interrupted".

Implementation. On every *best-checkpoint* save, the trainer now stores `optimizer.state_dict()`, `cosine_scheduler.state` (when present) and `scaler.state_dict()` (when AMP is on) inside the existing `extra` dict of the checkpoint payload. The schema version is unchanged. On resume, those state dicts are loaded inside try/except blocks so a checkpoint produced by an older code path still resumes cleanly (the cosine scheduler then falls back to its legacy fast-forward).

```
ckpt_extra: dict[str, Any] = {
    "val_top1": ckpt_stats.top1,
    "val_top5": ckpt_stats.top5,
    "val_loss": ckpt_stats.loss,
    "epoch": epoch + 1,
    "optimizer_state_dict": optimizer.state_dict(),
    "trained_class_indices": trained_class_indices,
    "checkpoint_kind": ckpt_kind,
}
if cosine_scheduler is not None:
    ckpt_extra["scheduler_state_dict"] = cosine_scheduler.state_dict()
if scaler is not None:
    ckpt_extra["scaler_state_dict"] = scaler.state_dict()
```

2.5 Trained-class indices and untrained-logit masking

Logic. The provided dataset has 33 unique class indices in $[0, 32]$ but class 27 has no training samples. The classifier head is nonetheless 33-wide (its weight tensor was initialised at random for class 27) and at inference time argmax can therefore land on class 27 by accident, producing systematically wrong predictions for every test clip on which the random class 27 logit happens to be the largest.

Implementation. The trainer derives the set of class indices that actually have at least one training sample and writes it into the checkpoint:

```
trained_class_indices: list[int] = sorted(
    {int(label) for _, label in train_samples})
```

The submission pipeline reads this list back and builds an additive mask that pushes every never-trained class to $-\infty$ before `argmax`:

```python
def _build_untrained_mask(trained_class_indices, num_classes, device):
    if not trained_class_indices:
        return None
    trained = {int(i) for i in trained_class_indices
               if 0 <= int(i) < num_classes}
    if len(trained) >= num_classes:
        return None
    mask = torch.zeros(num_classes, dtype=torch.float32, device=device)
    untrained = [c for c in range(num_classes) if c not in trained]
    mask[untrained] = -math.inf
    return mask
```

Older checkpoints that pre-date Phase 1 do not carry this list; in that case the mask is `None` and the legacy unmasked argmax path is used, preserving backward compatibility.

3 Architectural and Training Enhancements

Phase 2 adds four orthogonal opt-in features that compose cleanly with Phase 1. None of them change the behaviour of the legacy code paths when left at their defaults; together, they are bundled in a new experiment config `configs/experiment/track_a_phase2.yaml` that also flips on AdamW with a stronger weight decay.

3.1 Stochastic Depth (DropPath)

Logic. Stochastic Depth [2] is a stochastic regularisation technique that, during training, drops the residual branch of each ResNet bottleneck block with probability p_ℓ and rescales the kept contributions by $1/(1-p_\ell)$ so that the expectation over the random mask matches the no-drop forward pass:

$$y_\ell \;=\; x_\ell \;+\; \frac{b_\ell}{1-p_\ell}\,F_\ell(x_\ell), \qquad b_\ell \sim \text{Bernoulli}(1-p_\ell).$$

At inference time, all blocks are kept ($b_\ell = 1$ deterministically) so no rescaling is needed. The standard recipe uses a linear schedule that drops deeper blocks more aggressively:

$$p_\ell \;=\; p_{\max}\,\frac{\ell-1}{L-1},$$

where L is the total number of bottleneck blocks. For ResNet-50 $L = 16$ and a typical $p_{\max}$ is in $[0.05, 0.2]$.

Implementation. A parameter-free `DropPath` module implements the per-sample bernoulli mask. A helper `_inject_drop_p` walks the four ResNet stages, attaches a `DropPath` instance to each block (with the linearly-scaled probability), and rebinds the block's `forward` to a patched implementation that calls `self.drop_path(out)` on the residual branch immediately before the residual addition:

```python
def _patched_bottleneck_forward(self, x: torch.Tensor) -> torch.Tensor:
    identity = x
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.relu(self.bn2(self.conv2(out)))
    out = self.bn3(self.conv3(out))
    if self.downsample is not None:
        identity = self.downsample(x)
    out = self.drop_path(out) # <-- new
    return self.relu(out + identity)
```

`DropPath` adds no parameters and no buffers, so the model's `state_dict` layout is unchanged regardless of `model.drop_path_rate`. This is asserted by a unit test that compares state-dict key sets with and without DropPath enabled.

3.2 Attention-pool head

Logic. The legacy temporal head averages the per-frame features:

$$z = \frac{1}{T}\sum_{t=1}^{T}\phi(x_t), \qquad \widehat{y} = Wz + b.$$

A 1-query multi-head attention pool is a strict generalisation: when the attention weights are uniform it reproduces the mean exactly, but it can also concentrate on the most informative frame(s). Concretely, we introduce a learnable query token $q \in \mathbb{R}^{1\times D}$ and compute

$$z \;=\; \mathrm{LayerNorm}(\mathrm{MultiHeadAttn}(q,\, K{=}V{=}\Phi))\,,$$

where $\Phi \in \mathbb{R}^{T\times D}$ is the stack of frame features. With $T = 4$ and $D = 2048$, this adds about $D + 4D^2 \approx 16{,}780{,}288$ parameters, which is small compared to the ≈ 25M parameter ResNet-50 trunk.

Implementation. A new `AttentionPool` module wraps a single `nn.MultiheadAttention` layer plus a final `nn.LayerNorm`; the query token is initialised with a truncated normal of std 0.02. The supervised model exposes a new `model.head` switch (`"mean"` or `"attn"`) and routes features accordingly:

```
sequence_features = frame_features.view(batch_size, num_frames, -1)
if self.attn_pool is not None:
    consensus_features = self.attn_pool(sequence_features)
else:
    consensus_features = sequence_features.mean(dim=1)
```

When `model.head` = `"mean"` (the default) the `attn_pool` attribute is `None` and no extra parameters are added.

3.3 Exponential Moving Average (EMA) of model weights

Logic. Polyak averaging [4] maintains an exponentially-weighted average of the model's parameters,

$$\theta_{\mathrm{EMA}}^{(t+1)} \;=\; \alpha\,\theta_{\mathrm{EMA}}^{(t)} \;+\; (1-\alpha)\,\theta^{(t+1)}, \qquad \alpha \in [0.99, 0.9999],$$

which empirically gives smoother loss curves and a small but consistent top-1 boost ($\approx 0.3{-}1.0$ points) on most modern image and video classifiers.

Implementation. We use PyTorch's `torch.optim.swa_utils.AveragedModel` with a custom averaging function and `use_buffers=True` so BatchNorm running statistics are also tracked:

```
def _ema_avg_fn(avg_param, model_param, _num_averaged):
    return ema_decay * avg_param + (1.0 - ema_decay) * model_param

ema_model = torch.optim.swa_utils.AveragedModel(
    model, avg_fn=_ema_avg_fn, use_buffers=True).to(device)
```

`train_one_epoch` accepts a new `ema_model` keyword and calls `ema_model.update_parameters(model)` after every optimizer step. At the end of each epoch the trainer evaluates *both* the live and the EMA model and saves the better of the two:

```
if ema_stats is not None and ema_stats.top1 > val_stats.top1:
    ckpt_stats, ckpt_module, ckpt_kind = ema_stats, ema_model, "ema"
else:
    ckpt_stats, ckpt_module, ckpt_kind = val_stats, model, "live"

# ...later, before save_checkpoint:
module_to_save = (ckpt_module.module
                  if isinstance(ckpt_module,
                                torch.optim.swa_utils.AveragedModel)
                  else ckpt_module)
```

This selection rule is what lets downstream consumers (`submit.py`, `evaluate.py`) stay completely oblivious to the EMA toggle: the saved `model_state_dict` is always loaded with `build_model(...).load_state_dict(...)`, regardless of which copy won. The string `"live"` or `"ema"` is recorded in `extra["checkpoint_kind"]` for traceability.

3.4 Test-Time Augmentation with auto class-pair remap

Logic. Horizontal flip is a free *evaluation-time* augmentation *provided* that the predicted class is invariant to the flip. The Something-Something V2 subset used in this competition contains a small number of direction-sensitive classes, in particular:

- class 018: "Pulling something from left to right"
- class 019: "Pulling something from right to left"

For these, a flipped clip should predict the *paired* class. A naive flip-and-average TTA actually hurts accuracy on these two classes because it averages two confident but *disagreeing* predictions.

Implementation. We average softmax probabilities across the two views (original and flipped) and apply a learned permutation $\pi : \{0, \dots, K-1\} \to \{0, \dots, K-1\}$ to the flipped softmax so that paired classes are mapped back into agreement. The permutation is *auto-derived from the class folder names*, making it robust to any future change in the class set:

```
LR = "from_left_to_right"
RL = "from_right_to_left"
for idx, stem in stem_by_idx.items():
    if LR in stem:
        mirror_stem = stem.replace(LR, RL)
        for other_idx, other_stem in stem_by_idx.items():
            if other_stem == mirror_stem:
                perm[idx] = other_idx
                perm[other_idx] = idx
                break
```

(The numeric prefix is stripped before matching so `018_..._left_to_right` pairs with `019_..._right_to_left` without a code-level hard-code.) The TTA loop in `submit.py` then becomes:

```
logits = _logits_for_batch(model, video_batch, untrained_mask)
probs_total = torch.softmax(logits, dim=1)
n_views = 1
if tta_flip:
    flipped = torch.flip(video_batch, dims=[-1])
    flipped_probs = torch.softmax(
        _logits_for_batch(model, flipped, untrained_mask), dim=1)
    if flip_perm is not None:
        flipped_probs = flipped_probs.index_select(dim=1, index=flip_perm)
    probs_total = probs_total + flipped_probs
    n_views += 1
predictions.extend(int(p) for p in
    (probs_total / n_views).argmax(dim=1).cpu().tolist())
```

The configuration is opt-in (`training.tta`, `training.tta_flip`, both `false` by default) and read at inference time only; training is unaffected.

3.5 AdamW and the bundled experiment configuration

The trainer already supported AdamW; Phase 2 simply makes it the recommended optimizer for the new experiment by setting `training.optimizer: adamw` and `training.weight_decay: 0.05` in `configs/experiment/track_a_phase2.yaml`. Decoupling the weight-decay term from the gradient (which is what AdamW does, unlike plain Adam) is the right thing to do whenever a non-zero weight decay is used; it is also the optimizer of choice for ConvNeXt, ViT and modern ResNet recipes.

The `track_a_phase2.yaml` bundle composes all of Phase 1 and Phase 2:

```
defaults:
  - override /model: avanced_resnet50_tsm
  - override /augment: randaugment_t

model:
  drop_path_rate: 0.1
```

```
  head: attn
  head_num_heads: 4

dataset:
  num_frames: 4
  use_official_val: true

training:
  optimizer: adamw
  weight_decay: 0.05
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
  scheduler_cosine: true
  warmup_epochs: 10
  label_smoothing: 0.1
```

4 Self-Supervised Pretraining (V-JEPA)

The single highest-impact change available within Track A is to give the trunk something useful to do with the unlabeled clips before the supervised loss starts back-propagating. Track A permits self-supervised pretraining on the *provided* unlabeled images (the labels are simply not consumed and the trunk starts from random initialisation), so we pretrain on the union of the `train/`, `val/` and `test/` folders.

4.1 Why the DINO-on-still-frames prototype was deprecated

Our first SSL attempt was a DINO-v1 [3] pipeline trained on still frames sampled independently from the provided videos (file `src/smth2smth/shared/data/still_frames_dataset.py` and `src/smth2smth/shared/models/dino_ssl.py`). In practice it produced *inconsistent* downstream gains — sometimes a small improvement, sometimes a clear regression versus the random-init baseline. Two structural issues explain why:

1. **DINO is invariance-trained.** The objective is to assign the same prototype to two augmented views of the *same image*, which actively pressures the trunk to become invariant to the small per-pixel changes that distinguish adjacent video frames — exactly the signal an action-anticipation model needs to keep.
2. **The TSM modules are never exercised.** DINO operates on isolated frames, so the channel-shift modules wrapped around every bottleneck's `conv1` (which carry $\approx 12.5\%$ of the input channels temporally) see no temporal context during pretraining. Roughly one in eight conv1 input channels was therefore left at its Kaiming He initial value when supervised training started.

The DINO files remain in the repository as a historical record, but the recommended SSL pipeline is now the V-JEPA-style one described in the rest of this section.

4.2 V-JEPA-style clip-level pretraining

V-JEPA (Joint-Embedding Predictive Architecture for Video) [7, 8] replaces the "invariance to augmented views" objective by "predict the masked parts of a clip in feature space". Concretely, the student encodes a *masked* clip while the EMA-updated teacher encodes the unmasked clip; a small predictor maps the student's representation to the teacher's space, and an L1 loss is applied only at the masked positions. This is the right inductive bias for our problem: the student is forced to use temporal context to fill in masked features, which makes the TSM channel-shift modules immediately useful.

We adapt the recipe to our TSM-equipped ResNet-50 trunk with $T = 4$ frames. Because there are only 4 frames per clip, full V-JEPA 3D multi-block tube masking in patch-token space is not appropriate; we use *frame-level* masking instead, which mirrors the V-JEPA objective at the granularity that maps naturally onto our CNN trunk.

4.3 Clip dataset for SSL

A new file `src/smth2smth/shared/data/clip_ssl_dataset.py` exposes two utilities:

- `collect_all_video_dirs(roots)` walks the provided directories and returns every video folder it finds. It handles both layouts in use: class-bucketed (`train/<class>/<video>/<frame>.jpg`) and flat (`test/<video>/<frame>.jpg`). Duplicates across roots are removed.
- `ClipSSLDataset` yields one (T, C, H, W) clip per video. Frame indices are picked by the same `pick_frame_indices` helper used by supervised training so the SSL run sees exactly the same temporal distribution as the supervised trainer. The per-frame transform is the project's standard `build_transforms(is_training=True)`, which includes RandAugment-T spatial augmentations *without* ImageNet normalisation (Track A forbids pretrained weights).

On the question of V-JEPA's data augmentations. The V-JEPA paper combines random resized crop, horizontal flip, colour jitter, gray-scale and Gaussian blur in a specific stack. We *do not* reproduce that augmentation stack verbatim: we reuse the project's existing RandAugment-T pipeline so that the SSL trunk sees exactly the same input distribution as the downstream supervised trainer. Reproducing V-JEPA's stack would put a domain shift between pretraining and fine-tuning, which is the opposite of what we want. The *augmentation* contribution of V-JEPA that we *do* reproduce is the masking itself: n_{masked} frames per clip are replaced by zeros so the student must reconstruct their features from the unmasked context through the TSM channels.

4.4 Model: trunk + predictor + EMA teacher

`src/smth2smth/shared/models/vjepa_ssl.py` contains three pieces.

Trunk. `VJepaTrunk` is the *same* TSM-wrapped ResNet-50 as the supervised model's `backbone`: it calls `_make_temporal_shift` with `n_segment` $= T$, so every bottleneck block's `conv1` is replaced by `nn.Sequential(TemporalShift, conv1)`. The state-dict layout therefore matches `AvancedResNet50TSM.backbone.state_dict()` byte-for-byte, which lets the supervised trainer pick up the SSL checkpoint with a single `load_state_dict(..., strict=False)`.

Predictor. A small per-frame MLP (`Linear` → `GELU` → `Linear`, hidden dim 2048) maps the student's per-frame features into the teacher's space. The predictor exists so the encoder is not biased toward an identity student-to-teacher mapping on masked positions (Bardes et al., Sec. 3).

Composite. `VJepaModel` composes the trunk and the predictor. Two copies are instantiated at training time — a student and a teacher — with identical architecture; the teacher's weights are an EMA of the student's and are never updated by gradient descent.

4.5 Frame-level masking and the V-JEPA L1 loss

For each clip we sample $n_{\text{masked}} \sim \text{Uniform}[1, T-1]$ and zero out the corresponding frames. The student trunk encodes the masked clip; the teacher trunk encodes the unmasked clip with no gradient. The masked-position L1 loss is

$$\mathcal{L} = \frac{1}{|\mathcal{M}| \cdot D} \sum_{(b,t)\in\mathcal{M}} \left\| \hat{f}_{b,t} - f^{\text{tch}}_{b,t} \right\|_1 ,$$

where $\hat{f}_{b,t} = \text{Predictor}(\text{Student}(\tilde{x})_{b,t})$ is the predicted feature for clip b at frame t, $f^{\text{tch}}_{b,t} = \text{Teacher}(x)_{b,t}$ is the teacher's target, $\mathcal{M}$ is the set of masked (b,t) positions and $D = 2048$. Masked positions are the only ones that contribute to the loss; unmasked frames see no learning signal, which keeps the gradient focused on *predicting what is missing*.

```python
def vjepa_feature_loss(predicted, teacher, mask):
    if not mask.any():
        return predicted.sum() * 0.0 # zero with valid grad_fn
    diff = (predicted - teacher).abs() # (B, T, D)
    sel = mask.to(diff.dtype).unsqueeze(-1) # (B, T, 1)
    weighted = diff * sel
    n_elems = mask.sum().clamp_min(1).to(diff.dtype) * diff.shape[-1]
    return weighted.sum() / n_elems
```

4.6 Teacher EMA momentum schedule

The teacher's parameters are updated by $\theta_t \leftarrow m\,\theta_t + (1-m)\,\theta_s$ after every optimizer step, with m following a cosine schedule from $m_0 = 0.996$ at the start to $m_1 = 1.0$ at the end of training.

4.7 Trunk-only checkpoint format

After every epoch we save a small ($\approx$ 95 MiB) checkpoint that contains *only* the trunk parameters, with both the `trunk.` and the inner `backbone.` prefix stripped. The result is a flat dictionary keyed exactly like a standard `torchvision.models.resnet50` state-dict, except that the bottleneck `conv1` keys already carry the TSM-wrapping index (e.g. `layer1.0.conv1.1.weight` instead of `layer1.0.conv1.weight`):

```
trunk_state_dict = {}
for k, v in student.state_dict().items():
    if k.startswith("trunk.backbone."):
        trunk_state_dict[k.removeprefix("trunk.backbone.")] = v
torch.save({"trunk_state_dict": trunk_state_dict,
            "epoch": epoch + 1}, out_path)
```

4.8 The `model.init_from` hook in `train.py`

The supervised trainer received a single new opt-in:

```
init_from = cfg.model.get("init_from") if hasattr(cfg.model, "get") \
            else None
if init_from:
    payload = torch.load(Path(str(init_from)).resolve(),
                         map_location=device, weights_only=False)
    trunk_state = payload.get("trunk_state_dict")
    prefixed = _ssl_trunk_to_supervised_keys(trunk_state)
    missing, unexpected = model.load_state_dict(prefixed, strict=False)
```

The helper `_ssl_trunk_to_supervised_keys` handles both flavours of SSL checkpoint that the repository can produce:

- Legacy DINO trunks whose keys come from a plain `torchvision` ResNet-50 (e.g. `layer1.0.conv1.weight`). The helper renames them to the TSM-wrapped form (`layer1.0.conv1.1.weight`) and prepends `backbone.`.
- V-JEPA trunks whose keys are already TSM-wrapped (`layer1.0.conv1.1.weight`). The rename regex is a no-op and the helper only prepends `backbone.`.

```
block_conv1_re = re.compile(
    r"^(layer[1-4]\.\d+\.conv1)\.(weight|bias)$")
out: dict[str, torch.Tensor] = {}
for k, v in trunk_state.items():
    match = block_conv1_re.match(k)
    if match is not None:
        new_k = f"{match.group(1)}.1.{match.group(2)}"
    else:
        new_k = k
    out[f"backbone.{new_k}"] = v
return out
```

A V-JEPA trunk loads with **318 backbone tensors covered, zero unexpected keys, and only the classifier (and, when enabled, the attention pool) left at their fresh-init values** — the exact behaviour the test suite pins (`tests/pipelines/test_init_from_vjepa.py`).

5 Data Augmentation, Class Balance, and TTA Improvements

Phase 4 attacks the failure modes that remain once a good trunk is in place. Three of them stem directly from the small, imbalanced nature of the provided training set; the fourth is a free accuracy boost at inference time. All four are opt-in via the configuration and are bundled together in `configs/experiment/track_a_vjepa_finetune.yaml`.

5.1 Class imbalance: balanced sampling *and* balanced loss

Logic. The training set is heavily skewed: class counts range from ≈ 162 to ≈ 3170 samples, a 20× ratio. A vanilla cross-entropy loss with uniform sampling effectively *down-weights* rare classes twice over (they are seen less often, *and* each sample contributes equally to the loss), and the resulting classifier tends to predict the head classes for most test clips. We fix this on two independent axes.

Balanced sampler. We replace the uniform shuffle of the `DataLoader` by a `WeightedRandomSampler` whose per-sample weight is $w_i = 1/\sqrt{n_{c(i)}}$, where $c(i)$ is the class of sample i and n_c is the class count. The square root is a robust compromise between fully-balanced sampling (which heavily oversamples the 162-sample classes) and uniform sampling (which ignores them). The policy is selected by `training.class_balance_sampler` ∈ {none, inverse, sqrt_inverse}.

Balanced loss. We additionally weight each class in the cross-entropy by Cui et al.'s effective-number weights [9]: for class c with n_c samples,

$$w_c \;\propto\; \frac{1-\beta}{1-\beta^{n_c}}, \qquad \beta \in (0,1),$$

which interpolates between uniform weighting ($\beta \to 0$) and inverse-frequency weighting ($\beta \to 1$). We use the canonical $\beta = 0.999$. Classes with zero samples (e.g. class 27, see Section 2.5) receive $w_c = 0$, so they contribute neither to the sampling nor to the loss. The same weight vector is also routed through the soft-target cross-entropy used by label smoothing and FrameMixUp so the augmentations remain class-balanced.

```
# Per-class effective-number weights (Cui et al. 2019).
def _class_factors(n_per_class, policy, beta=0.999):
    if policy == "cb":
        effective = [(1.0 - beta**max(n, 1)) / (1.0 - beta)
                     for n in n_per_class]
        return [1.0 / e if e > 0 else 0.0 for e in effective]
    # ... 'inverse', 'sqrt_inverse' branches ...
```

The two axes are configured independently (`training.class_balance_sampler`, `training.class_balance_loss`, `training.c` so the user can run any combination of {sampler off, sampler sqrt} × {loss off, loss CB} and compare. The new `class_balance.py` module is fully unit-tested (`tests/shared/utils/test_class_balance.py`).

5.2 Motion-aware time-reversal augmentation

Logic. For verbs whose semantics are intrinsically direction-sensitive, the time-reversed clip belongs to a *paired* class — the reverse of opening is closing, the reverse of pushing left-to-right is pushing right-to-left, etc. Time-reversal is therefore a label-preserving augmentation *only* when we also remap the label to the paired class. Without the remap, a reversed clip injects flat-out wrong supervision.

Implementation. A new module `src/smth2smth/shared/data/time_reversal.py` hosts a small hand-curated table of reversible class pairs:

> Opening ↔ Closing, Moving up ↔ Moving down, Pushing left-to-right ↔ Pushing right-to-left, Tilting toward came
> Tilting away from camera, plus a symmetric set (e.g. *Holding something*) where the reverse is the same class.

`build_time_reversal_table` matches the pairs against the actual class folder names on disk and returns two tensors: `perm[c]` = the class index of the time-reversed clip (`perm[c] = -1` if c is not reversible), and `allow_mask[c] = True` iff class c is reversible. Both are threaded into `VideoFrameDataset` via three new keyword arguments:

```
class VideoFrameDataset(Dataset):
    def __init__(self, ..., *,
                 time_reversal_prob: float = 0.0,
                 time_reversal_perm: torch.Tensor | None = None,
                 time_reversal_allow_mask: torch.Tensor | None = None):
        ...
    def __getitem__(self, index):
        ...
        if self._should_reverse(label):
```

```
            indices = list(reversed(indices))
            label = int(self.time_reversal_perm[int(label)].item())
        ...
```

The augmentation is gated by `dataset.time_reversal_prob` (default 0.0). When set to e.g. 0.3, about 30% of clips drawn from reversible classes are reversed and re-labelled at every epoch, which roughly doubles the effective training set for the affected classes without any synthetic data.

5.3 Multi-scale TTA and logit adjustment

The Phase 2 TTA (Section 3.4) already averages horizontal-flip predictions with an auto-derived left/right class permutation. Phase 4 extends inference with two further tricks.

Multi-scale TTA. At inference time we resample the input clip to several spatial scales (e.g. $\{0.875, 1.0, 1.125\} \times H \times W$) using bilinear interpolation, run the model on each scale (optionally with a horizontal flip), softmax each set of logits, and average all softmaxes. This is a standard test-time ensembling trick for CNN classifiers; it is gated by `training.tta_scales` (a list, default [1.0]).

```
def _rescale_video(video_batch, scale):
    if abs(scale - 1.0) < 1e-6:
        return video_batch
    b, t, c, h, w = video_batch.shape
    new_h, new_w = int(round(h * scale)), int(round(w * scale))
    return F.interpolate(video_batch.reshape(b * t, c, h, w),
                         size=(new_h, new_w),
                         mode="bilinear",
                         align_corners=False).view(b, t, c, new_h, new_w)
```

Logit adjustment. Even with a class-balanced sampler and loss, the predictor at test time still inherits a small bias toward head classes whenever the training data and the test data have different (but unknown) class distributions. Menon et al. [10] show that subtracting $\tau \cdot \log \pi_c$ from class c's logit at *inference time*, where π_c is the training-set prior and $\tau \in [0, 1]$ is a small constant, is the Bayes-optimal post-hoc correction under the standard balanced-test assumption. We compute π_c directly from the training-folder counts and subtract $\tau \log \pi_c$ from the logits right after the forward pass, before either the softmax or the multi-scale averaging:

```
def _build_logit_adjustment(train_dir, num_classes, tau, device):
    counts = class_counts(...)
    total = float(sum(counts))
    pi = torch.tensor([c / total if c > 0 else 1e-12 for c in counts],
                      device=device)
    return tau * torch.log(pi) # subtracted from logits

# inside _logits_for_batch:
logits = model(video_batch)
if logit_adjust is not None:
    logits = logits - logit_adjust
```

The strength τ is configured via `training.tta_logit_adjust` (default 0.0 = off; we use $\tau = 1.0$ in the bundled experiment).

5.4 The bundled experiment configuration

The four Phase-4 features compose cleanly with everything in Phases 1 through 3. A new experiment file `configs/experiment/` bundles them on top of a V-JEPA warm-start:

```
defaults:
  - override /model: avanced_resnet50_tsm
  - override /augment: randaugment_t

model:
  drop_path_rate: 0.1
  head: attn
```

```
  head_num_heads: 4
  init_from: null # set on the command line

dataset:
  num_frames: 4
  use_official_val: true
  time_reversal_prob: 0.3

training:
  optimizer: adamw
  weight_decay: 0.05
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
  tta_scales: [0.875, 1.0, 1.125]
  tta_logit_adjust: 1.0
  class_balance_sampler: sqrt_inverse
  class_balance_loss: cb
  class_balance_beta: 0.999
  early_stopping_enabled: true
  early_stopping_patience: 20
```

6 Configuration and reproducibility

6.1 New configuration knobs

The following table summarises every new switch introduced by Phases 1 through 4. All defaults preserve the legacy behaviour.

Key	Default	Effect when on
Phases 1–2		
`dataset.use_official_val`	`false`	validate on `data/val/`
`training.amp`	`false`	FP16 autocast + GradScaler
`training.ema_enabled`	`false`	EMA of model weights
`training.ema_decay`	`0.999`	EMA momentum
`training.tta`	`false`	enable TTA at submit time
`training.tta_flip`	`true`	flip view (only used when `tta`)
`training.tta_temporal_shift`	`0`	extra temporal start-offsets (TTA)
`model.drop_path_rate`	`0.0`	Stochastic Depth maximum drop prob.
`model.head`	`mean`	temporal head (`mean`/`attn`)
`model.head_num_heads`	`4`	MultiHead attention heads
`model.init_from`	`null`	path to SSL trunk to warm-start
Phase 3 (V-JEPA SSL)		
`pretrain.mask_prob`	`0.5`	avg. frame mask fraction for V-JEPA
`pretrain.min_mask`	`1`	min frames masked per clip
`pretrain.max_mask`	`2`	max frames masked per clip
`pretrain.predictor_hidden_dim`	`2048`	MLP predictor hidden width
`pretrain.teacher_momentum_base`	`0.996`	cosine EMA momentum start
`pretrain.teacher_momentum_final`	`1.0`	cosine EMA momentum end
Phase 4		
`training.class_balance_sampler`	`none`	sample weights: `none/inverse/sqrt_inverse`
`training.class_balance_loss`	`none`	class-balanced loss policy (`none/cb`)
`training.class_balance_beta`	`0.999`	β in the CB effective-number formula
`dataset.time_reversal_prob`	`0.0`	prob. of time-reversing a reversible-class clip
`training.tta_scales`	`[1.0]`	list of spatial scales for multi-scale TTA
`training.tta_logit_adjust`	`0.0`	τ in Menon et al. logit adjustment

6.2 New experiment configurations

`configs/experiment/track_a_phase2.yaml`
Bundles every Phase-1 and Phase-2 opt-in: AdamW + WD 0.05, AMP, DropPath 0.1, attention head, EMA 0.999, TTA at submit time, official validation split, cosine schedule with 10-epoch warmup, label smoothing 0.1, RandAugment-T, FrameMixUp.

`configs/experiment/track_a_phase2_balanced.yaml`
Phase 2 *plus* all of Phase 4 (class-balanced sampler + CB loss, time-reversal augmentation, multi-scale TTA, logit adjustment) but *without* SSL warm-start. Useful as an ablation baseline.

`configs/experiment/track_a_vjepa_pretrain.yaml`
Routes the run through `pretrain_vjepa.py`: 50 epochs of V-JEPA-style clip mask-denoising over the union of train+val+test videos, 1–2 frames masked per clip, AdamW + WD 0.04, AMP on.

`configs/experiment/track_a_vjepa_finetune.yaml`
Same as `track_a_phase2_balanced.yaml` but expects `model.init_from` to be set on the command line to the V-JEPA trunk produced by the previous experiment; reduces the number of supervised epochs (warm-started runs converge faster) and halves the learning rate.

`configs/experiment/track_a_ssl_pretrain.yaml, ..._ssl_finetune.yaml`
The legacy DINO-on-still-frames experiments are retained for reference. They are superseded by the V-JEPA variants above and are not recommended for new runs (see Section 4.1).

6.3 Backwards compatibility

Every legacy entry point continues to work without modification:

- Phase-1 checkpoints (no `trained_class_indices`, no `optimizer_state_dict` in `extra`) load and resume via the legacy fast-forward fallback.
- Pre-Phase-1 checkpoints (no Phase-2 opt-ins enabled) load with identical state-dict keys; this is enforced by a unit test that compares state-dict key sets between defaults and Phase-2 opt-ins.
- Submission CSVs produced before Phase 1 remain valid; the new mask is only applied if the checkpoint actually carries the trained-class list.

7 Test coverage

The pre-existing suite contained 120 tests, all of which still pass. Forty-one new tests were added across the four phases, bringing the total to **161 passing tests**:

`tests/shared/models/test_phase2_optins.py` **(8 tests)**
Phase 2: `DropPath` eval-time identity and unbiased per-sample training behaviour; `DropPath(0)` byte-for-byte equality with the no-op; attention pool shape; state-dict key invariance under `drop_path_rate`; `attn_pool.*` key appearance.

`tests/pipelines/test_submit_tta.py` **(5 tests)**
Phase 2: auto-derived left/right class permutation; no-TTA path; untrained-mask path; flip-TTA softmax reinforcement of the correct class after remap.

`tests/shared/data/test_still_frames_dataset.py` **(6 tests)**
Phase 3 (DINO, retained): frame-path discovery across both folder layouts; deduplication across roots; multi-view sampling shape.

`tests/shared/models/test_dino_ssl.py` **(4 tests)**
Phase 3 (DINO, retained): SSL trunk key superset of the supervised backbone keys; finite back-propagating loss; teacher-EMA midpoint identity.

`tests/pipelines/test_init_from_ssl.py` (1 test)
Phase 3 (DINO, retained) end-to-end smoke test for the `model.init_from` hook.

`tests/shared/models/test_vjepa_ssl.py` (10 tests)
Phase 3 (V-JEPA): trunk forward shape; remapped V-JEPA trunk keys cover the supervised backbone *exactly* (zero missing, zero unexpected); `load_state_dict` only misses the classifier; the L1 loss is zero for an empty mask (with a valid `grad_fn`) and finite-and-backprop-able otherwise; `apply_frame_mask` zeroes the correct positions; mask sampler bounds and "never-all-masked" guarantee; teacher EMA midpoint.

`tests/shared/data/test_clip_ssl_dataset.py` (7 tests)
Phase 3 (V-JEPA): both folder layouts; cross-root deduplication; missing-root skip; sample shape; temporal jitter under hold-out.

`tests/pipelines/test_init_from_vjepa.py` (1 test)
Phase 3 (V-JEPA) end-to-end smoke test that explicitly pins the no-double-`backbone.`-prefix invariant of the V-JEPA save format.

`tests/shared/utils/test_class_balance.py`,
`tests/shared/data/test_time_reversal.py`,
`tests/pipelines/test_submit_phase4_tta.py`
Phase 4: per-class weight policies (`none`, `inverse`, `sqrt_inverse`, `cb`) and their corner cases (zero-count class set to weight 0); time-reversal table-building and label-remap symmetry (pair-permuted twice = identity); multi-scale TTA shape and logit-adjustment numerics.

8 Expected Impact

8.1 Per-change order-of-magnitude estimates

Based on the literature and on the gap currently observed between internal validation and the public leaderboard, we expect the following contributions to Track A's public Top-1 score (these are order-of-magnitude estimates, not promises):

Change	Internal val	Public Top-1
Baseline (current best)	0.5666	0.3584
+ official val split (Phase 1)	lower *	–
+ Kaiming init (Phase 1)	+0.5–2 pt	+0.3–1 pt
+ AMP (Phase 1)	speedup only	speedup only
+ optimizer/scheduler state on resume (Phase 1)	0–1 pt	0–1 pt
+ untrained-class masking (Phase 1)	negligible	+0.5–1.5 pt
+ DropPath (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ Attention head (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ EMA model averaging (Phase 2)	+0.3–0.8 pt	+0.2–0.6 pt
+ TTA flip with class-pair remap (Phase 2)	+0–0.5 pt	+0.5–1 pt
+ AdamW + WD 0.05 (Phase 2)	+0.3–1 pt	+0.3–1 pt
+ DINO SSL pretraining (deprecated)	–1 to +1 pt †	–1 to +1 pt †
+ V-JEPA SSL pretraining (Phase 3)	+2–4 pt	+2–4 pt
+ Class-balanced sampler + CB loss (Phase 4)	+1–2 pt	+1–2 pt
+ Time-reversal augmentation (Phase 4)	+0.5–1.5 pt	+0.5–1.5 pt
+ Multi-scale TTA (Phase 4)	+0.3–0.8 pt	+0.3–0.8 pt
+ Logit adjustment (Phase 4)	+0.2–0.5 pt	+0.2–0.5 pt

* The official split is harder than the leaky internal split, so reported val will drop — but the public score will move closer to the val score.
† As observed in practice and explained in Section 4.1: DINO on still frames produces inconsistent downstream gains. We retain the row in this table only as a baseline against which V-JEPA's stable gains stand out.

9 Legacy DINO Pretraining (Deprecated)

The single highest-impact change available within Track A is to give the trunk something useful to do with the unlabeled frames before the supervised loss starts back-propagating. We implemented DINO-v1 [3], a teacher-student knowledge-

distillation framework that has consistently outperformed contrastive methods (MoCo, SimCLR) on ResNet-50 trunks at the dataset sizes we are working with.

9.1 Track-A compliance

The competition forbids *external data* and *pretrained models*. Self-supervised pretraining on the *provided* unlabeled images is therefore allowed: the labels are simply not consumed and the trunk starts from random initialisation. We pretrain on the union of the `train/`, `val/` and `test/` folders, which gives roughly $\sim$ 12,000 still frames ($\sim$ 3,000 videos $\times$ 4 frames) and is sufficient to drive the DINO loss for many hundreds of steps per epoch at batch size 64.

9.2 Still-frames dataset

A new file `src/smth2smth/shared/data/still_frames_dataset.py` exposes two utilities:

- `collect_all_frame_paths(roots)` walks the provided directories. It transparently handles both layouts in use: `train/<class>/<video>/<frame>.jpg` (via `collect_video_samples`) and `test/<video>/<frame>.jpg` (one level shallower, no class bucket). Duplicate paths are removed.
- `MultiViewStillFramesDataset` returns, for each frame, two global views (typically 224×224 with random crop scale in $[0.4, 1.0]$) and L local views (typically 96×96 with random crop scale in $[0.05, 0.4]$). The augmentations include horizontal flip, colour jitter, grayscale, and Gaussian blur, following the DINO recipe.

The dataset returns a `dict` with keys `global_views` and `local_views`; a custom collate function stacks them into per-view batches.

9.3 The DINO model

`src/smth2smth/shared/models/dino_ssl.py` contains three pieces.

Trunk. `build_resnet50_trunk()` returns a `torchvision.models.resnet50(weights=None)` with its final `fc` replaced by `nn.Identity`. The trunk emits 2048-d features.

Head. `DinoHead` is the canonical 3-layer projection MLP used in DINO: `Linear` → `GELU` → `Linear` → `GELU` → `Linear` (to a low-dimensional bottleneck) → L2 normalisation → weight-normalised `Linear` mapping to K prototypes ($K = 4096$ by default).

Composite. `DinoModel` composes the trunk and the head. Two copies are instantiated at training time — a *student* and a *teacher*. Both have identical architecture; the teacher's weights are an EMA of the student's and are never updated by gradient descent.

9.4 The DINO loss

Let $\mathcal{V} = \{v_1, \ldots, v_{2+L}\}$ be the set of all views of one image (two globals, L locals). Let $p_s(v)$ and $p_t(v)$ be the softmax-normalised outputs of the student and teacher at temperatures τ_s and τ_t respectively. The loss is

$$\mathcal{L} \;=\; \frac{1}{|\mathcal{P}|} \sum_{(t,s)\in\mathcal{P}} H(p_t(v_t),\; p_s(v_s))\,,$$

where $\mathcal{P}$ is the set of (teacher view, student view) pairs with *distinct* indices and H is the standard cross-entropy. The teacher only processes the two global views (so $|\mathcal{P}| = 2 \cdot |\mathcal{V}| - 2 \cdot 1$), but the student processes *all* views. The teacher's logits are sharpened ($\tau_t = 0.04$ is much smaller than $\tau_s = 0.1$) and centred by an EMA of the batch mean to prevent collapse:

$$c^{(t+1)} \;=\; m_c\, c^{(t)} \;+\; (1 - m_c)\, \frac{1}{B} \sum_{i=1}^{B} \mathrm{logits}_t(v_i^{\mathrm{global}}),$$

with momentum $m_c = 0.9$.

```
class DinoLoss(nn.Module):
    def forward(self, student_logits_per_view, teacher_logits_per_global):
        teacher_probs_per_global = [
            F.softmax((logits - self.center) / self.teacher_temp,
                      dim=-1).detach()
            for logits in teacher_logits_per_global]
        student_log_softmax_per_view = [
            F.log_softmax(logits / self.student_temp, dim=-1)
            for logits in student_logits_per_view]
        total_loss, n_terms = 0.0, 0
        for t_idx, t_probs in enumerate(teacher_probs_per_global):
            for s_idx, s_log in enumerate(student_log_softmax_per_view):
                if t_idx == s_idx:
                    continue
                total_loss = total_loss \
                             + (-(t_probs * s_log).sum(dim=-1).mean())
                n_terms += 1
        # ... centering buffer EMA update ...
        return total_loss / n_terms
```

9.5 Teacher EMA momentum schedule

The teacher's parameters are updated by $\theta_t \leftarrow m\,\theta_t + (1-m)\,\theta_s$ after every optimizer step, with m following a cosine schedule from $m_0 = 0.996$ at the start to $m_1 = 1.0$ at the end of training. This mirrors the recipe in the DINO paper and stabilises training significantly — a constant momentum is more prone to collapse on small datasets.

9.6 Trunk-only checkpoint format

After every epoch we save a small ($\approx$ 95 MiB) checkpoint that contains *only* the trunk parameters, with the `trunk.` prefix stripped:

```
trunk_state_dict = {
    k.removeprefix("trunk."): v
    for k, v in student.state_dict().items()
    if k.startswith("trunk.")
}
torch.save({"trunk_state_dict": trunk_state_dict,
            "epoch": epoch + 1}, out_path)
```

This format is intentionally minimal: there is no point keeping the DINO head, the teacher copy or the optimizer state once supervised training takes over.

9.7 The `model.init_from` hook in `train.py`

The supervised trainer received a single new opt-in:

```
init_from = cfg.model.get("init_from") if hasattr(cfg.model, "get") \
            else None
if init_from:
    payload = torch.load(Path(str(init_from)).resolve(),
                         map_location=device, weights_only=False)
    trunk_state = payload.get("trunk_state_dict")
    prefixed = _ssl_trunk_to_supervised_keys(trunk_state)
    missing, unexpected = model.load_state_dict(prefixed, strict=False)
```

The non-trivial part is the key remap. The supervised `AvancedResNet50TSM` wraps every bottleneck block's `conv1` in

$$\texttt{block.conv1} \;\leftarrow\; \texttt{nn.Sequential(TemporalShift(}T\texttt{, shift\_div), conv1)},$$

which means the parameter that the SSL trunk has at key `layer1.0.conv1.weight` now lives at `layer1.0.conv1.1.weight` in the supervised model. `_ssl_trunk_to_supervised_keys` performs this rename unconditionally for every key matching `layer[1-4]\.\d+\.conv1\.(weight|bias)`:

```
block_conv1_re = re.compile(
    r"^(layer[1-4]\.\d+\.conv1)\.(weight|bias)$")
out: dict[str, torch.Tensor] = {}
for k, v in trunk_state.items():
    match = block_conv1_re.match(k)
    if match is not None:
        new_k = f"{match.group(1)}.1.{match.group(2)}"
    else:
        new_k = k
    out[f"backbone.{new_k}"] = v
return out
```

The load is intentionally non-strict so that the supervised model's classifier head and (when enabled) attention pool keys remain at their fresh-init values. The trainer logs how many tensors were loaded, and warns loudly if any backbone key was *not* covered by the SSL checkpoint.

9.8 Dual-stream RGB + frame-difference TSM

Beyond the single-stream `AvancedResNet50TSM` baseline, the codebase includes `dual_stream_rgb_diff_tsm`: two backbones with TSM inserted in the usual bottleneck (ResNet-50 on RGB frames) and basic-block (ResNet-34 on consecutive RGB differences $I_t - I_{t-1}$) branches. Per time index $t = 0, \ldots, T-1$ the model forms a fused token by concatenating the RGB embedding at t with a motion embedding—zero-padded at t=0, otherwise the difference embedding for $(t-1, t)$—and projecting to a fixed width (`model.fuse_dim`, default 512). A shared temporal readout (mean or 1-query attention, matching the single-stream head options) and linear classifier produce clip logits. See `docs/diagrams/dual_stream_rgb_diff_attention.svg` for a block diagram of the architecture.

Configuration. Model defaults live in `configs/model/dual_stream_rgb_diff_tsm.yaml`. A ready-made Track A training recipe (30 epochs, warmup, RandAugment-T + FrameMixUp, official validation split, AMP, EMA, class boosting with deterministic paired-verb duplicate rows, validation every epoch via `training.eval_every_n_epochs=1`) is `configs/experiment/track_a_dual_stream_30e_class_boost.yaml`. Stochastic `temporal_reversal_augment` is disabled automatically when `dataset.class_boosting.enabled` is true. The README documents the Hydra invocation.

SSL warm-start. The `model.init_from` hook from §4.8 applies only to `AvancedResNet50TSM` backbone key names; training with `dual_stream_rgb_diff_tsm` skips loading so that mis-matched prefixes are not applied silently.

9.9 Per-backbone learning rates for the dual-stream model

Empirically, training the RGB ResNet-50 and the motion ResNet-34 with the *same* learning rate is suboptimal: the motion branch sees a noisier input signal (consecutive-frame difference $I_t - I_{t-1}$ rather than raw RGB), has fewer parameters, and tends to overfit before the deeper RGB trunk has converged. Two new keys in `configs/train/default.yaml` expose a per-branch peak LR without changing legacy behaviour:

- `training.motion_lr` — absolute peak LR for the ResNet-34 motion branch. Overrides the ratio when both are set.
- `training.motion_lr_ratio` — relative multiplier; the motion branch peak LR is lr × ratio. Default `null` keeps the historical single-LR-group path.

When either knob is set and the model is the dual-stream builder, `pipelines/train.py` constructs three optimiser groups (`rgb_backbone`, `motion_backbone`, `fuse_head`) with per-group `warmup_lr_max`, so linear warmup and cosine annealing both respect the per-branch peak LR. The shared `min_lr` is reused as the cosine $\eta_{\min}$ for all groups. This path is mutually exclusive with the LoRA two-group path (LoRA is not used by the dual-stream model, so the choice is well-defined).

Long-run preset. The companion experiment YAML `configs/experiment/track_a_dual_stream_long_class_boost.yaml` extends the 30-epoch class-boost preset with: an 80-epoch budget so cosine has room to anneal, `motion_lr_ratio: 0.3`, aggressive early stopping (`early_stopping_patience: 3`, $\Delta = 0.001$), and a separate checkpoint path. The bash launcher `scripts/run_dual_stream_long_with_fallback.sh` chains a slower-LR fallback: if Run 1's log contains `Early`

`stopping triggered`, it launches a second run with `lr=0.6` × $\texttt{lr}_1$ (default `6e-5`) into a fresh log/checkpoint pair, otherwise it exits cleanly.

Result. Run A1 was stopped externally after epoch 16 (31.79 % EMA val top-1). Run A1b resumed from the `*.last.pt` checkpoint with a 20 % higher RGB/fuse LR (`lr`=1.2e-4, `resume_apply_cfg_lr`=1), trained through epoch 39, and triggered early stopping (patience 3). Best EMA validation top-1: **39.47** % (`ckpt: .../dual_stream_long_class_boost_lr1_2026` Flip-TTA CSV `.../track_a_dual_stream_long_lr1_resume_20260515.csv` (6 913 test clips) scored **37.69** % public top-1 on the Kaggle leaderboard—∼ 1.9 pp above the previous Track-A best (35.84 %, Tab. in §1) and within ∼ 1.8 pp of the official validation proxy (39.47 %), consistent with the official-val fix from Phase 1.

9.10 Track A symbol glossary (training runs)

Mirrors the Track-B convention from §10.5; symbols are reused where they exist in both tracks so a reader does not have to juggle two conventions. Compact assignments below are the ones used in the Track-A run logbook (Tab. 2).

Sym.	Hydra key(s)	Meaning
`expt`	`experiment=...`	Preset YAML under `configs/experiment/`.
`T`	`dataset.num_frames`	Frames per clip.
`Sz`	`dataset.image_size`	Square crop side.
`bs`	`training.batch_size`	Clips per optimiser step.
`E`	`training.epochs`	Total epoch budget.
`lr`	`training.lr`	Base / RGB-branch peak LR (AdamW).
`m_r`	`training.motion_lr_ratio`	Motion-branch peak LR as a fraction of lr; `null` for legacy single-group.
`wd`	`training.weight_decay`	AdamW weight decay.
`wu`	`training.warmup_epochs`	Linear LR warm-up length.
`cos`	`training.scheduler_cosine`	Cosine anneal after warm-up.
`ls`	`training.label_smoothing`	Soft-label smoothing for CE.
`amp`	`training.amp`	Mixed precision.
`ema`	`training.ema_enabled`	EMA of weights; decay in `ema_decay`.
`es`	`training.early_stopping_patience`	Patience (val top-1 evals without improvement) before early stop.
`cb`	`dataset.class_boosting.enabled`	Deterministic paired-verb row duplication.
`aug`	augment group	Spatial/temporal augment preset.
`ckpt`	`training.checkpoint_path`	Primary weights file.

Table 1: Symbols used in Tab. 2. Extra CLI overrides are appended verbatim after a semicolon.

9.11 Track A training run logbook

Statuses: **RUNNING** (PID alive or log still progressing), **DONE** (trainer printed `Done.`), **FAILED** (traceback or non-zero exit). Update the corresponding row when a job finishes; add new rows for every fresh `nohup` launch.

9.12 Supervised VideoMAE-style ViT from scratch: verdict

Every supervised `video_mae_vit` run above—ViT-B vs ViT-S, mean pool vs attentive probe, with or without class boosting / balancing—stayed near ∼ 1/33 random chance scaled by a small factor (∼ 5–8 % train top-1, ∼ 3–6 % val top-1). The attentive probe (Run A4) did *not* unlock learning: mid-training logs still show loss ≈ ln(33) and ∼ 6.5 % train accuracy, indistinguishable from mean pool (Run A2). Class tricks (rouget / roussette / raie) marginally move numbers but remain ≪ the ∼ 39 % val regime of the dual-stream CNN (Runs A1/A1b). **Conclusion:** training this ViT *from random init* on ∼ 45 k labelled clips alone is not viable for Track A; the practical path is **VideoMAE self-supervised pretraining** on provided frames (`track_a_videomae_pretrain`) followed by supervised fine-tune (`track_a_videomae_finetune` with `model.init_from`), as configured in this repository.

The overnight **SSL fish-matrix** (slots 8–16, `SSL.md`) implements this path per codename. Slot 10 (**roussette**) completed end-to-end with best EMA val **33.61 %** (seed/batch drift vs YAML—see `experiments.md`). Run A5 (**truite**, slot 9) finished SSL pretrain only; slot 12 (**raie**) best EMA val **34.80 %** (6-head probe); slot 15 (**piranha**) best EMA val **35.60 %** (40 FT epochs); slot 11 (**rouget**) completed end-to-end with honest val ∼ 35 %, public test ∼ 36 % (Tab. 2). A follow-up that trained on `val` clips as well reported ∼ 60 % val but *worse* generalisation—see `experiments.md` and the rouget row above. Slot 8 (**barbeau**) was not run.

Table 2: Chronological log of substantial Track-A training jobs.

Run A1 (STOPPED)
Wrapper: `scripts/run_dual_stream_long_with_fallback.sh`. Log: `logs/track_a_dual_stream_long_lr1_20260515_025557`. Preset `track_a_dual_stream_long_class_boost` (Tab. 1); `aug`=randaugment_t; `T`=4; `Sz`=224; `bs`=16; `E`=80; `lr`=1e-4; `m_r`=0.3 (⇒ motion lr=3e-5); `wd`=0.05; `wu`=6; `cos`=1; `ls`=0.1; `amp`=1; `ema`=0.999; `es`=3 (Δ = 0.001); `cb`=1. Stopped externally mid epoch 17. Best EMA val top-1 (epoch 16): **31.79 %**.

Run A1b (DONE)
Log: `logs/track_a_dual_stream_long_lr1_resume_20260515_104715.log`. Resume from `*.last.pt` (epoch 16); override `lr`=1.2e-4 (+20%), `m_r`=0.3, `resume_apply_cfg_lr`=1; same preset otherwise. Early stop at epoch 39 (patience exhausted). Best EMA val top-1: **39.47 %**; same `ckpt` as Run A1. Submit (flip TTA): `.../track_a_dual_stream_long_lr1_resume_20260515.csv`; log `.../track_a_dual_stream_long_lr1_resume_20260515_submit.log`. Public Kaggle top-1: **37.69 %**.

Run A2 / truite (DONE – FAILED LEARNING)
Log: `logs/truite.log`. Preset `track_a_vit_truite`: video_mae_vit `variant`=vit_b, `head`=**mean** (mean pool, MP), ∼87 M params; same train recipe as the fish-matrix presets below. `T`=4; `Sz`=224; `bs`=16; `E`=40; `lr`=5e-4; `wu`=5; RandAug-T (`n`=4, `m`=7); cube CutMix; `cb`=0. Early stop after 18 epochs; best val top-1 **5.41 %**; train top-1 plateaued near ∼ 7 % (loss ln 33 ≈ 3.5).

Run A3 / sardine (STOPPED – FAILED LEARNING)
Log: `logs/track_a_vit_sardine_20260515_230200.log`. Preset `track_a_vit_sardine`; `variant`=vit_s; MP; ∼22 M; `bs`=64; `lr`=1e-3; `wu`=10; RandAug-T (`n`=6, `m`=10). Stopped manually after epoch 3: ∼ 6.7 % train / **6.15 %** best val (epoch 2)—same failure mode as truite despite 4× larger batch and heavier aug.

Run A4 / truite AP (STOPPED – FAILED LEARNING)
Log: `logs/track_a_vit_truite_ap_20260515_234500.log`. Preset `track_a_vit_truite_ap`: ViT-B with **attentive probe** (`head=attn`, 4 heads), ∼89 M params; otherwise identical to truite. Stopped manually mid-training (same plateau as MP): e.g. mid-epoch steps still show train loss ≈ 3.35 and train top-1 ≈ 6.5–7 %—*no gain* from replacing mean pool. Confirms the bottleneck is not the temporal readout.

Run A5 SSL / truite (slot 9) (PRETRAIN DONE; FT NOT RUN)
Phase 1 VideoMAE SSL (`smth2smth.pipelines.pretrain_videomae`); preset `track_a_ssl_pretrain_truite` (ViT-S baseline, no `SSL.md` §7 overrides). Log: `logs/ssl_truite_pretrain.log`. Launcher: `.venv/bin/python` with `PYTHONPATH=src` (`nohup`; minimal shells often lack `uv` on `PATH`). MAE `E`=100, `wu`=5, `bs`=16, `lr`=1.5e-4, `mask`=0.75; ∼ 58 651 unlabeled clips; avg loss 0.851 → 0.256 (ep 100); encoder `checkpoints/track_a/ssl/truite_encoder.pt` (last epoch, rewritten each epoch). **Phase 2 not run:** no `ssl_truite_finetune.log`, no `truite_ft.pt` (pending `track_a_ssl_finetune_truite` after encoder). Distinct from supervised Runs A2/A4 (`track_a_vit_*`).

Run rouget (ViT-B + MP + class boosting; DONE – FAILED)
Preset `track_a_vit_rouget`: truite stack + deterministic class boosting (`cb`=1). Representative log (epoch 16): train ∼ 8.6 % top-1, val ∼ 4.6 %, best val still only **5.59 %** with patience counting down—no useful convergence.

Run roussette (ViT-B + MP + class balancing; DONE – FAILED)
Preset `track_a_vit_roussette`: sqrt-inverse sampler + CB loss, no boosting. Early stop at epoch 23; best val top-1 **5.23 %**.

Run raie (ViT-B + MP + boosting + balancing; DONE – FAILED)
Preset `track_a_vit_raie`: both `cb` and balancing. Best val top-1 **6.00 %**; example epoch 15: val ∼ 6.1 % live / ∼ 5.9 % EMA with train top-1 still ∼ 5.7–5.8 %—long-tail tricks do not substitute for convolutional inductive bias or SSL features.

SSL slot 8 / barbeau (NOT RUN)
Thomas slot: ViT-B, MAE `warmup_epochs`=10 (`track_a_ssl_pretrain_barbeau` / `track_a_ssl_finetune_barbeau`). Job was not launched.

SSL slot 12 / raie (DONE)
Romain slot: attentive probe `head_num_heads`=6.

Phase 1. 100/100 MAE after one AMP `scatter_` crash (retry); loss 0.851 → 0.254; `raie_encoder.pt`; logs `ssl_raie_pretrain.log` / `.crash.log`.

Phase 2. 30/30 via `chain_raie.sh`; official val only; train top-1 **44.15 %**; best EMA val **34.80 %** (ep 29, `raie_ft.pt`); log `ssl_raie_finetune.log`. Failed post-sweep resume (OOM) excluded. No submission.

SSL slot 10 / roussette (DONE)
Romain slot: YAML `seed`=123 (§7); resolved run used `seed`=42; pretrain `bs`=8 vs planned 16 (CLI).

Phase 1. 100/100 MAE; loss 0.843 → 0.251; `checkpoints/track_a/ssl/roussette_encoder.pt`; log `logs/ssl_roussette_pretrain.log`.

Phase 2. 30/30 FT; train top-1 **41.44 %** (ep 30); best EMA val top-1 **33.61 %** (ep 27, saved); final val **33.62 %** live /

10 Track B: Open-World with V-JEPA 2

Track B (*Open World*) allows pretrained backbones and external data. The highest-impact lever in this regime is therefore the choice of backbone itself: a strong, self-supervised *video* encoder will dominate any per-frame ResNet trained from scratch. We use **V-JEPA 2** [8], Meta FAIR's 2025 video JEPA that reports state-of-the-art results on motion understanding benchmarks and, of direct relevance here, the strongest published Something-Something v2 numbers among open weights: 77.3% top-1 with the ViT-g/16 384 encoder plus an attentive probe (Tab. 3).

Benchmark	V-JEPA 2	Previous best
SSv2 (probe)	**77.3%**	69.7% (InternVideo2-1B)
Diving48 (probe)	**90.2%**	86.4% (InternVideo2-1B)
EK100 (probe)	**39.7%**	27.6% (PlausiVL)

Table 3: V-JEPA 2 attentive-probe results reported in [8]. SSv2 is the source distribution of the challenge data set: a frozen V-JEPA 2 encoder almost matches the previous best *fine-tuned* number.

10.1 Track-B compliance and design choices

Track B explicitly allows pretrained weights and external data, so using the publicly released V-JEPA 2 checkpoints (trained by Meta on internet-scale video) is permitted. We choose the HuggingFace integration over the upstream PyTorch Hub one because the `transformers.AutoModel` path (i) ships a well-tested `VJEPA2Model` class, (ii) caches checkpoints under `$HF_HOME` so we can swap variants by name, and (iii) gives access to every publicly released V-JEPA 2 variant from a single API:

HF repository	Arch.	Params	Res.
`facebook/vjepa2-vitl-fpc64-256`	ViT-L	0.3 B	256
`facebook/vjepa2-vith-fpc64-256`	ViT-H	0.7 B	256
`facebook/vjepa2-vitg-fpc64-256`	ViT-g	1.0 B	256
`facebook/vjepa2-vitg-fpc64-384`	ViT-g	1.0 B	384
`facebook/vjepa2-vitl-fpc16-256-ssv2`	ViT-L	0.4 B	256
`facebook/vjepa2-vitg-fpc64-384-ssv2`	ViT-g	1.0 B	384

The two `*-ssv2` variants come with an SSv2-finetuned encoder and are particularly interesting for our 33-class subset: the features have already been adapted to the Something-Something distribution, so the probe converges in a handful of epochs.

Why a frozen probe. Following the standard V-JEPA evaluation protocol [8], the encoder is frozen and only the classifier probe is trained. This:

- Caps trainable parameters at $\sim$1 M (rather than 0.3–1 B), which makes the run fit comfortably on a single GPU with batch size 4–8 even at the 384-resolution ViT-g.
- Avoids destroying the unsupervised representation by overfitting on a 33-class slice of 50 k clips.
- Reproduces a published number (77.3 % on the full SSv2) instead of inventing a new training recipe.

10.2 The `vjepa2` model

A new file `src/smth2smth/track_b/vjepa2.py` defines three pieces.

Trunk. `VJEPA2Probe` composes `AutoModel.from_pretrained(hf_repo)` (the V-JEPA 2 encoder) with a trainable head. The encoder is loaded with `attn_implementation="sdpa"` so PyTorch's scaled-dot-product kernel is used (FlashAttention-2 when available; eager fallback otherwise). When `freeze_backbone=true` and **no** LoRA (Phase 4.4), the forward path is

```python
def forward(self, video_batch: torch.Tensor) -> torch.Tensor:
    if self._encoder_needs_grad:
        tokens = self._encode(video_batch)
    else:
        with torch.no_grad():
            tokens = self._encode(video_batch)
    return self.head(tokens)
```

With `lora_enabled=true`, `_encoder_needs_grad` is true: the pretrained weights stay frozen (via PEFT) but LoRA adapter paths receive gradients, so the `torch.no_grad` block is *not* used on the encoder forward.

`video_batch` has the shape (B, T, C, H, W) produced by `VideoFrameDataset` with ImageNet normalisation. That layout matches the HuggingFace `VJEPA2Model.forward` contract exactly (its `pixel_values_videos` is documented as "`(batch_size, num_frames, num_channels, frame_size, frame_size)`"), so no reshape is needed.

Frozen-mode invariants. `VJEPA2Probe.train(mode)` keeps the backbone in `eval()` when it is *fully* frozen (no LoRA), so dropout inside the encoder stays off:

```python
def train(self, mode: bool = True) -> VJEPA2Probe:
    super().train(mode)
    if self.freeze_backbone and not self.lora_enabled:
        self.backbone.eval()
    return self
```

When `lora_enabled=true`, the PEFT-wrapped backbone follows the parent `train()/eval()` so LoRA dropout behaves as intended during optimisation.

A unit test (`test_train_keeps_frozen_backbone_in_eval_mode`) asserts that, without LoRA, after `model.train()` the head is in train mode but the backbone remains in eval mode. A second test (`test_frozen_forward_does_not_accumulate_bac`) asserts that, after backprop with a fully frozen trunk, every backbone parameter has `p.grad is None` while every head parameter has non-trivial gradient mass (LoRA-enabled runs instead accumulate adapter gradients only).

Head. Three head types are offered.

`attentive` (default).
The V-JEPA paper's attentive probe, *generalised* to $Q \geq 1$ learnable query tokens $q \in \mathbb{R}^{Q \times D}$. Each query cross-attends over the (B, N, D) token sequence; the Q pooled vectors are **mean-pooled** across queries, then LayerNorm and a linear classifier are applied:

$$Z = \mathrm{MHA}(q,\, K{=}V{=}\Phi) \in \mathbb{R}^{B \times Q \times D}, \qquad z = \tfrac{1}{Q} \sum_{i=1}^{Q} Z_{:,i,:}, \qquad \widehat{y} = Wz + b.$$

The config key `head_num_queries` controls Q; the default $Q{=}1$ recovers the original single-query recipe. Larger Q adds probe capacity for compositional verbs without widening the backbone.

`linear` / `mean_linear`.
Mean-pool the token sequence and project with a single Linear. Fewer parameters, marginally faster; useful as an ablation baseline.

Builder. A `@register_model("vjepa2")` decorator exposes the model under the same registry the rest of the project uses:

```python
@register_model("vjepa2")
def build_vjepa2(cfg: DictConfig) -> nn.Module:
    return VJEPA2Probe(
        num_classes=int(cfg.model.num_classes),
        hf_repo=str(cfg.model.get("hf_repo", DEFAULT_HF_REPO)),
        head_type=str(cfg.model.get("head_type", "attentive")),
        head_num_heads=int(cfg.model.get("head_num_heads", 8)),
        head_num_queries=int(cfg.model.get("head_num_queries", 1)),
        head_dropout=float(cfg.model.get("head_dropout", 0.0)),
        freeze_backbone=bool(cfg.model.get("freeze_backbone", True)),
```

```
        lora_enabled=bool(cfg.model.get("lora_enabled", False)),
        lora_r=int(cfg.model.get("lora_r", 8)),
        lora_alpha=int(cfg.model.get("lora_alpha", 16)),
        lora_dropout=float(cfg.model.get("lora_dropout", 0.05)),
        lora_target_modules=...,
        attn_implementation=str(cfg.model.get("attn_implementation", "sdpa")),
    )
```

The ellipsis stands for optional `lora_target_modules` parsed from Hydra (default `null` ⇒ HF V-JEPA 2 `query`, `key`, `value`, `proj`).

10.3 Registration hook

`shared/models/__init__.py` imports `smth2smth.track_b` at module load time so that the `@register_model("vjepa2")` decorator runs before `build_model(cfg)` is ever called. This is the only cross-package import between `shared/` and `track_b/`; the opposite direction is empty (`track_b/vjepa2.py` only imports from `shared.models.registry`), so there is no circularity:

```
% src/smth2smth/shared/models/__init__.py
from smth2smth import track_b as _track_b % noqa: F401
```

A unit test (`test_vjepa2_is_registered`) asserts that `"vjepa2"` is in `list_registered_models()` after a clean import. A second test (`test_build_model_dispatches_via_registry`) goes one step further and runs `build_model(cfg)` with a mocked HuggingFace backbone (`sys.modules["transformers"]` is replaced with a stub) to check that the dispatch works end-to-end through the public API.

10.4 The bundled experiment configuration

`configs/experiment/track_b_vjepa2.yaml` composes the V-JEPA 2 model with sensible Track-B training defaults:

```
defaults:
  - override /model: vjepa2

model:
  pretrained: true # ImageNet normalisation
  hf_repo: facebook/vjepa2-vitl-fpc64-256
  head_type: attentive
  head_num_heads: 8
  freeze_backbone: true

dataset:
  num_frames: 16 # must be even (tubelet_size=2)
  image_size: 256 # match the backbone's crop_size
  use_official_val: true

training:
  batch_size: 4
  epochs: 30
  lr: 0.001
  optimizer: adamw
  weight_decay: 0.05
  scheduler_cosine: true
  warmup_epochs: 2
  label_smoothing: 0.1
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
```

Three settings are non-negotiable:

1. `model.pretrained: true`.
   `build_transforms` reads this and applies the ImageNet mean and std, which is what V-JEPA 2 was trained with. Setting it to `false` would normalise with $(0.5, 0.5, 0.5)$ and silently give wrong inputs to the encoder.

2. `dataset.image_size: 256` (or 384 for the `vitg-384` variants).
   The V-JEPA 2 encoders are tied to a fixed crop size by their learned positional embeddings; resizing inputs to anything else is not supported by the public weights.

3. `dataset.num_frames` must be even.
   The V-JEPA 2 tubelet size is 2; an odd T is not handled by the encoder's temporal patchifier and raises a runtime error.

The remaining settings (AMP, EMA, TTA, label smoothing, cosine schedule with 2-epoch warmup) carry over from the Phase-2 recipe with no changes, because the supervised `train.py` is generic over `cfg.model.name`.

10.5 Hydra symbol glossary (training runs)

Every long training job we care about is summarised in Tab. 5. Hyperparameters are written in **compact form** using the symbols below; the full English meaning lives here once, not repeated in each run row.

Sym.	Hydra key(s)	Meaning
expt	`experiment=...`	Preset YAML under `configs/experiment/` (composed defaults).
repo	`model.hf_repo`	HuggingFace V-JEPA 2 checkpoint id.
T	`dataset.num_frames`	Frames per clip (must be even).
Sz	`dataset.image_size`	Square crop side (256 or 384).
bs	`training.batch_size`	Clips per optimisation step.
E	`training.epochs`	Total epoch budget for the run.
lr	`training.lr`	Base learning rate (AdamW).
wd	`training.weight_decay`	AdamW weight decay.
wu	`training.warmup_epochs`	Linear LR warm-up length.
cos	`training.scheduler_cosine`	Cosine anneal after warm-up.
ls	`training.label_smoothing`	Soft-label smoothing for CE.
amp	`training.amp`	Automatic mixed precision.
ema	`training.ema_enabled`	Exponential moving average of weights (0/1); decay in `ema_decay`.
ev	`training.eval_every_n_epochs`	Validate every n epochs.
mz	`dataset.max_samples_per_class`	Cap clips per class (`null` = use all).
aug	`defaults` / augment group	Spatial/temporal augment preset (e.g. `vjepa2_heavy`).
LoRA	`model.lora_enabled`, `lora_r`, `lora_alpha`	PEFT adapters on attention projections; pretrained weights stay frozen.
ckpt	`training.checkpoint_path`	Primary weights file.
rs	`training.resume_from`	Optional optimiser/epoch resume path.

Table 4: Symbols used in Tab. 5. Extra CLI overrides are appended verbatim after a semicolon.

10.6 Track B training run logbook

Statuses: **RUNNING** (PID alive or log still progressing), **DONE** (trainer printed `Done.`), **FAILED** (traceback or non-zero exit). After any merge or local pull, refresh this table from `logs/` and PID files.

Table 5: Chronological log of substantial Track-B training jobs. Update status and metrics when processes finish; add new rows for every fresh `nohup` launch.

Run 1 (DONE)
Log: `logs/track_b_vitg384_heavy_aug_20260512_0159.log`. Preset: `track_b_vjepa2_vitg384_heavy_aug`. Params (Tab. 4): `repo`=vitg-fpc64-384-ssv2; `T`=16; `Sz`=384; `bs`=4; `E`=10; `lr`=3e-4; `wd`=0.05; `wu`=1; `mz`=100; `ev`=5; `ema`=0. Best val top-1: **53.68 %**.

Run 2 (DONE)
Log: `logs/track_b_vitg384_continue_20260512_1322.log`; PID: `logs/track_b_vitg384_continue.pid`. Preset: `track_b_vjepa2_vitg384_continue`. Params: inherits Run 1 + `rs`=vitg384_heavy_aug.last.pt; `E`=20 (epochs 11–20); `wu`=0; `ev`=1; `cos`=0. Best val top-1: **53.94 %**.

Run 3 (RUNNING)
Log: `logs/track_b_vitl_30e_warmup_lora_20260514_1536.log`; PID: `logs/track_b_vitl_30e_warmup.pid`. Preset: `track_b_vjepa2_vitl_30e_warmup`. Params: `repo`=vitl-fpc64-256; `T`=16; `Sz`=256; `bs`=4; `E`=30; `wu`=3; `ev`=1; `ema`=0; LoRA `r`=8, α=16. Interim (epoch 1) val top-1: **60.21 %**.

10.7 Running V-JEPA 2 on Track B

The sole additional dependency is `transformers` ≥ 4.45, the release that ships `VJEPA2Model`; training, evaluation and submission all go through the existing `smth2smth.pipelines` entrypoints with `track=b` and the appropriate experiment config. The encoder is downloaded from HuggingFace Hub on first use (≈ 1.2 GiB for `vitl-fpc64-256`) and cached under `$HF_HOME`; subsequent runs reuse the cache.

10.8 Zero-shot Evaluation with the SSv2 Head

The SSv2-finetuned checkpoint `facebook/vjepa2-vitl-fpc16-256-ssv2` ships a full 174-class classifier head trained by Meta on the full SSv2 distribution. Our 33-class subset is a strict subset of those 174 classes, so we can evaluate – and submit – with no training of our own. Two small artifacts implement this in the codebase:

- `src/smth2smth/track_b/zero_shot.py` – reusable helpers: `normalize_class_name`, `build_label_mapping` (with a token-aligned unique-prefix fallback that recovers the truncated folder `015_Pretending_to_pour_something_out_of_somet` and `VideoFramesDataset` that reads 16 evenly-spaced frames per clip, resizes to 256, ImageNet-normalises, and stacks to $(T, 3, H, W)$.
- `scripts/eval_vjepa2_ssv2_zero_shot.py` – CLI that loads `VJEPA2ForVideoClassification`, slices the 174-d logits to the matched 32 classes, and reports top-1, top-5, and a per-class accuracy table.
- `scripts/submit_vjepa2_ssv2_zero_shot.py` – same forward, but consumes `data/test/` and writes `submissions/track_b_` in the official two-column format; the file is re-validated with `validate_submission_csv` before exit. An optional `-tta-flip` flag mirrors the horizontal-flip TTA + auto left/right class-pair remap that `pipelines/submit.py` performs for trained checkpoints.

Result. On the official `data/val/` split (6 745 clips, 32 of 33 classes – class 27 has no validation data) the zero-shot top-1 is **44.0**% with top-5 at **74.5**%, in 373 s on a single NVIDIA RTX A4000 (18 vid/s). The public Kaggle leaderboard score on the 6 913-video test set is **45**% top-1 – already ~ 8 pp ahead of the Track-A from-scratch baseline (37.2%). The result is documented in detail in `notebooks/06_vjepa2_feature_geometry.ipynb` together with PCA and t-SNE visualisations of the encoder's pre-classifier features, which confirm that many of the 32 classes form visually well-separated clusters in 2D even before any training of our own — predicting that a thin probe should comfortably beat the zero-shot baseline.

10.9 Data Augmentation for the V-JEPA 2 Probe

The default `track_b_vjepa2.yaml` above only composes the model group; the augmentation group falls back to `augment=none` (resize + horizontal flip). That is a deliberate choice for the zero-shot evaluation, but for any training run we want to inject as much randomness as possible: the V-JEPA 2 encoder is frozen (only $\sim$ 4 M probe parameters receive gradients) so the probe will overfit our 33-class slice in a handful of epochs unless we vary the features it sees per epoch. Heavy augmentation does exactly that.

Mirroring Track A. The Track-A side of the project (`track_a_randaugment_t.yaml`) already implements Kim et al. 2020's best recipe (Tab. 7): RandAugment-T (`mode: temporal_plus`) layered with FrameMixUp. The same infrastructure transfers to Track B without code changes – both `shared/data/transforms.py` (per-frame augmentations) and `shared/engine/trainer.py` (clip-level VideoMix) are agnostic to the model name and the track. The only thing missing is the Track-B experiment that composes these knobs in the right place.

What `augment=vjepa2_heavy` stacks. `configs/augment/vjepa2_heavy.yaml` (kept from earlier exploration) chains, in order, with all randomness *synchronised across the frames of a clip* so that motion direction is preserved:

1. **RandAugment-T** (Kim et al. 2020, Sec. 3.1+4.2) – 14-op palette, N=2, M=9, `magnitude_max=30`, `mode: temporal_plus` which is the strongest setting in Tab. 2: a magnitude M is drawn per clip and the per-frame magnitude varies linearly from $M - \delta$ to $M + \delta$ with $\delta \sim U(0, 0.5M)$. This injects temporally-coherent geometric/photometric noise that mimics camera motion and lighting drift.
2. **Random crop** $256 + 32 \rightarrow 256$, mild zoom jitter that prevents the SSv2 prior from latching onto absolute pixel positions.

3. **Random horizontal flip**. The class-pair remap for "pulling left → right" vs "right → left" lives in the submission TTA path, not at train-time – the flip here just adds variety.
4. **Color jitter** (`brightness=0.3`, `contrast=0.3`, `saturation=0.3`, `hue=0.05`) on top of RandAugment.

Clip-level VideoMix. On top of the spatial augmentation, the Track-B heavy-aug experiment also enables one of the spatio-temporal mix strategies plumbed in `trainer.py`: we default to `videomix_mode: cube_cutmix` with `videomix_alpha: 1.0` and `videomix_prob: 0.5`. CubeCutMix [**?**] samples a 3-D spatio-temporal sub-volume from a second clip in the same batch and pastes it onto the current clip; labels are mixed in proportion to the sub-volume's relative volume. This is the regularizer of choice for motion-sensitive datasets (Kim et al. 2020 Tab. 7), where naive MixUp blurs motion cues across the entire clip.

Slim validation loop. `pipelines/train.py` historically runs two full validation passes per epoch (live + EMA), each at 18 vid/s on the A4000 → 12 minutes for the 6 745-clip val split. With heavy aug + 15 epochs that adds up to ∼3 h of pure-eval wall time, dwarfing the actual training. Two new Hydra knobs in `configs/train/default.yaml` gate this without breaking any earlier experiment:

```
training:
  eval_every_n_epochs: 1 # default: eval every epoch (legacy)
  eval_ema: true # default: also evaluate the EMA snapshot
```

With `eval_every_n_epochs: 3` and `eval_ema: false` we shrink the per-epoch overhead from 12 min to 4 min averaged over the schedule, which is exactly the optimisation the per-epoch timing run (Sec. 10.9) flagged as the dominant cost.

The bundled `track_b_vjepa2_heavy_aug` experiment. `configs/experiment/track_b_vjepa2_heavy_aug.yaml` composes the V-JEPA 2 SSv2 backbone with the heavy aug, CubeCutMix, the slim validation loop, and a small-subset training schedule that fits a single run in about two hours on the A4000:

```
defaults:
  - override /model: vjepa2
  - override /augment: vjepa2_heavy

model:
  pretrained: true
  hf_repo: facebook/vjepa2-vitl-fpc16-256-ssv2
  head_type: attentive
  head_num_heads: 8
  head_dropout: 0.1
  freeze_backbone: true

dataset:
  num_frames: 16
  image_size: 256
  use_official_val: true
  max_samples_per_class: 50

training:
  batch_size: 12
  epochs: 15
  lr: 0.0005
  optimizer: adamw
  weight_decay: 0.05
  scheduler_cosine: true
  warmup_epochs: 2
  min_lr: 0.00001
  label_smoothing: 0.1
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  eval_every_n_epochs: 3
  eval_ema: false
  videomix_mode: cube_cutmix
  videomix_alpha: 1.0
```

```
videomix_prob: 0.5
tta: true
tta_flip: true
```

Why this is the "smallest" V-JEPA 2. The smallest V-JEPA 2 SSv2-finetuned checkpoint available on HuggingFace Hub is `vitl-fpc16-256-ssv2` (0.4 B parameters). V-JEPA 2.1 ViT-B (80 M, distilled from ViT-G) was announced by Meta in March 2026 [**?**] but is not yet uploaded to the Hub at the time of writing; a tracking PR on `transformers` adds inference support, and once weights are public swapping `model.hf_repo` is the only change required. For now ViT-L 256 is both the smallest SSv2-finetuned encoder and also the encoder behind the 44 % zero-shot floor, so we keep it as the iteration baseline.

Why we did not add extra positional encoding. V-JEPA 2 already includes a learned 3-D positional embedding per token (T_{tubelet}=2 along time × $H/16$ × $W/16$ in space, 2 048 tokens for 16 frames at 256 px). The attentive probe's cross-attention reads from this positionally-aware token sequence with a learnable query, so the global pooling already has full positional context. Stacking another positional embedding on top would be redundant. The ceiling we hit at the zero-shot baseline is *informational*, not positional: `data/train/` only contains four unique frames per video, repeated 4× to fill 16 tubelet-aligned positions, so motion-direction-only classes (closing vs opening, moving-up vs moving-down) lose information at the source. Mitigations on this front (frame interpolation, optical flow channel) are deferred to a later phase.

10.10 Overnight ViT-g/14 384 Run

Motivation. The Phase 4.2 ViT-L 256 run topped out at 53.09 % val top-1 (checkpoint `checkpoints/track_b/vjepa2_heavy`, 15 epochs, 200 clips/class, EMA off, no augmentation). That is only +8.7 percentage points over the 44 % zero-shot floor and *below* the > 70 % anchor reported by peer submissions. The bottleneck is not augmentation strength but *backbone capacity*: a 0.4 B-parameter ViT-L probed on 6 400 clips is now competing with 1 B-parameter encoders trained on the same task. The Phase 4.3 run swaps to the largest SSv2-finetuned V-JEPA 2 available on the Hub – `facebook/vjepa2-vitg-fpc64-384-ssv2` (1.0 B parameters, 384 px input, 77.3 % SSv2 in the paper) – and combines it with the Phase 4.2 augmentation recipe. To keep wall time inside an overnight slot we shrink the training subset accordingly.

Memory and throughput smoke check. On the lab A4000 (16 GiB total, 12.2 GiB free at launch) a forward pass with `torch.float16` and `skip_predictor=True` measures 2.27 GiB resident after weight load and the following inference-only costs:

- batch 1, 16 frames @ 384 px: 882 ms/clip (≈ 1.13 vid/s), 2.49 GiB peak.
- batch 2: 1 548 ms/step (1.29 vid/s), 2.70 GiB peak.
- batch 4: 3 148 ms/step (1.27 vid/s), 3.10 GiB peak.
- batch 6: 4 553 ms/step (1.32 vid/s), 3.52 GiB peak.

Throughput plateaus at ∼1.3 vid/s – the encoder is memory-bandwidth-bound, so larger batches mostly reduce host-side overhead. We pick batch 4 to keep the optimiser stable while staying well below the 12 GiB free budget.

Recipe. The experiment lives at `configs/experiment/track_b_vjepa2_vitg384_heavy_aug.yaml` and overrides Phase 4.2 with the following changes:

- `model.hf_repo: facebook/vjepa2-vitg-fpc64-384-ssv2` (frozen, ∼1.0 B params, ∼8.0 M trainable in the attentive probe).
- `dataset.num_frames: 16`, `image_size: 384`.
- `dataset.max_samples_per_class: 100` (100×32 = 3 200 train clips, balanced).
- `training.batch_size: 4`, `epochs: 10`, cosine LR with 1 warm-up epoch, lr=3×10^{-4}, wd=0.05, label smoothing 0.1.

- `augment=vjepa2_heavy` (RandAugment-T temporal_plus + random crop + flip + colour jitter) and clip-level CubeCutMix (α=1.0, p=0.5).
- Slim validation: `eval_every_n_epochs: 5` (only 2 val passes total, epochs 5 and 10), `ema_enabled: false` so no ∼4 GiB deepcopy.
- TTA flip + class-pair remap at submission time.

Resume-safe checkpointing. A long run that crashes between val checkpoints can lose hours of work: the previous trainer only wrote `best_model.pt` on validation improvements. We added two new `training.*` knobs (see `configs/train/default.yaml`):

- `save_last_checkpoint: true` – end of every epoch the live model state, optimiser state, cosine scheduler state and AMP `GradScaler` state are written to a companion file.
- `last_checkpoint_path: null` – by default derived from `checkpoint_path` by inserting `.last` before the extension (`vitg384_heavy_aug.pt` → `vitg384_heavy_aug.last.pt`).

On a crash, restart with

```
PYTHONPATH=src python -m smth2smth.pipelines.train \
  track=b experiment=track_b_vjepa2_vitg384_heavy_aug \
  training.checkpoint_path=checkpoints/track_b/vitg384_heavy_aug.pt \
  training.resume_from=checkpoints/track_b/vitg384_heavy_aug.last.pt
```

The trainer reads back the saved epoch counter, restores optimiser/scheduler/scaler state, and resumes at the next epoch boundary. The existing `tests/pipelines/test_last_checkpoint.py` unit-tests pin all three contract points (write-every-epoch, opt-out, override path).

Launch command. Detached, with a timestamped log file and a PID file for monitoring:

```
PYTHONPATH=src nohup .venv/bin/python -u \
  -m smth2smth.pipelines.train \
  track=b experiment=track_b_vjepa2_vitg384_heavy_aug \
  training.checkpoint_path=checkpoints/track_b/vitg384_heavy_aug.pt \
  hydra.run.dir=outputs/track_b_vitg384_heavy_aug_<stamp> \
  > logs/track_b_vitg384_heavy_aug_<stamp>.log 2>&1 < /dev/null &
echo $! > logs/track_b_vitg384_heavy_aug.pid
```

`nohup` keeps the run alive after the shell disconnects; `-u` forces unbuffered stdout so the log file reflects progress in real time; `< /dev/null` stops the worker from blocking on `stdin`.

Wall-time projection (overnight slot). With 800 training steps per epoch (3 200 clips / batch 4) and the smoke-measured ∼ 1.3 vid/s effective training throughput (encoder forward dominates; backward only flows through the 8 M-param probe), one epoch costs ≈ 41 min, observed to be ≈ 45 min in the actual log including data-loader overhead. The full `data/val/` pass costs ∼78 min at the same throughput. Total budget: $10 \times 45 + 2 \times 78 \approx 606$ min, i.e. ∼10 h – fits a 01:59 → 12:00 window.

Outcome. Finished ViT-g 384 heavy-aug jobs are summarised in Tab. 5 (rows 1–2) with compact notation from Tab. 4. Phase 4.3 supersedes Phase 4.2 as the flagship Track-B recipe for large backbones; the submission CSV is produced via `python -m smth2smth.pipelines.submit` pointing at `checkpoints/track_b/vitg384_heavy_aug.pt`.

10.11 Multi-query Probe and PEFT LoRA

Motivation. The gap between course val (∼54% with ViT-g 384) and the paper's 77.3 % SSv2 benchmark is only partly model-size: the frozen encoder cannot adapt its internal features to the 33-class slice or to domain shift, while a single-query attentive probe may under-represent compositional verbs. Phase 4.4 adds two orthogonal extensions implemented in `src/smth2smth/track_b/vjepa2.py` and exposed through `configs/model/vjepa2.yaml`:

1. **Multi-query attentive head** — config key `head_num_queries` (default `1`). Q learnable queries each run cross-attention over the full token grid; outputs are mean-pooled across Q before `LayerNorm` and the classifier. This increases probe capacity without touching the ViT trunk.
2. **PEFT LoRA** — optional low-rank adapters (LoRA, Hu et al., ICLR 2022) on attention linear projections (`query`, `key`, `value`, `proj` by default for the HuggingFace `VJEPA2Model` encoder; configurable via `lora_target_modules`). The `peft` package wraps the HuggingFace encoder with `get_peft_model`; pretrained weights stay frozen while adapter matrices train. Forward passes run *with* autograd through the adapters (`_encoder_needs_grad=true`), unlike the pure frozen trunk. Full encoder fine-tuning remains available via `freeze_backbone=false` with `lora_enabled=false` (not combinable with LoRA in the current implementation).

Dependencies. The project declares `peft>=0.13` in `pyproject.toml`. LoRA is inactive unless `model.lora_enabled=true`.

Experiment preset. `configs/experiment/track_b_vjepa2_peft.yaml` composes the Phase 4.3 ViT-g 384 heavy-aug recipe with `head_num_queries: 4`, `lora_enabled: true`, `lora_r: 8`, `lora_alpha: 16`, and a slightly reduced learning rate (`training.lr: 2e-4`) so adapter updates remain stable. Checkpoints are written to `checkpoints/track_b/vitg384_peft` by default.

Caveats. If PEFT cannot match `lora_target_modules` to the encoder's module names, construction raises a clear `RuntimeError`; adjust the list for the specific `hf_repo`. QLoRA (4-bit base weights) is *not* implemented here—only BF16/FP16 training as before. Dense multi-crop / multi-clip test-time ensembling remains future work.

11 Conclusion

Track A. The pipeline foundations work removes silent correctness issues that inflated validation scores and establishes a reliable training baseline. The architectural enhancements layer DropPath, an attention-pool head, EMA, and flip-TTA on top of the existing RandAugment-T + FrameMixUp recipe. Self-supervised pretraining documents both legacy DINO-on-still-frames (deprecated) and V-JEPA-style clip SSL for the TSM trunk, plus the VideoMAE SSL overnight sweep (∼35 % official-val top-1 for the best ViT-S slots; dual-stream CNN at ∼39.5 % val / 37.7 % public remains the strongest CNN baseline). Data augmentation and TTA improvements address class imbalance and inference-time logit adjustment. Operational results and config drift are summarised in `experiments.md` and Tab. 2.

Track B. The open-world track centres on V-JEPA 2 with probe tuning, heavy augmentation, and optional PEFT; see Tab. 5 for chronological runs.

All changes are opt-in via Hydra; legacy checkpoints and entry points remain compatible where tested.

A Inventory, Verification, and Recommendations

A.1 Executive summary

- **Best public Track-A submission to date**: a Phase-2 single-stream `avanced_resnet50_tsm` run from scratch under `configs/experiment/track_a_phase2.yaml` with a 150-epoch cosine schedule, attentive head, EMA, and multi-scale flip-TTA at submit reached ≈ **42** % public Top-1 on an earlier run whose checkpoint was lost. The active re-run `logs/trackA_phase2_randaugT_attn_dropPath0.1_adamw_ema_amp_4frames_officialVal_ep150_20260516.log` is at epoch 38/150 with EMA val 0.3936 and climbing monotonically; see §A.3. This recipe supersedes the dual-stream Run A1b (EMA val 39.47 %, public 37.69 %) as the leading single-model path.
- **Best on-disk honest val from the VideoMAE SSL sweep**: *piranha* **35.60** % EMA (40 FT epochs, ep 32), then *rouget* 34.96 % (Phase 2a honest), *raie* 34.80 % (6-head probe), *roussette* 33.61 %. **None have been submitted.** Honest val here means `dataset.use_official_val=true` *and* the absence of any `dataset.include_val_in_train` override, verified from each finetune log.
- **Only Kaggle SSL data point**: *rouget* 36.0 % public from `rouget_ft.pt` *after* Phase 2b (leaky: train+val labels both in training). Aligns with the Phase 2a honest val of 34.96 %, not the inflated ∼ 60 % Phase 2b internal — training on validation *did not* beat the dual-stream submission (§9.11).
- **Supervised `video_mae_vit` from scratch**: every variant (*truite*, *sardine*, *truite AP*, *rouget*, *roussette*, *raie*) plateaued at ≤ 6 % val on ∼ 45 k labelled clips (§9.12). The audit confirms the verdict: no further runs in this family are warranted.
- **Local checkpoint asymmetry**: only `checkpoints/track_a/ssl/rouget_encoder.pt` and the leaky `rouget_ft.pt` are present on this VM. The piranha / raie / roussette / truite encoders and FT weights referenced by `experiments.md` live on the producing VMs and must be retrieved before any submission. The Run A1b checkpoint `dual_stream_long_class_boost_lr1_20260515_025557.pt` is likewise absent from `checkpoints/track_a/`; the current `best_model.pt` (∼ 1 GiB, dated 15 May 23:37) is consistent in size with the dual-stream model but its identity must be confirmed prior to any resubmission.
- **Multi-scale TTA caveat for VideoMAE**: `submit.py`'s `_rescale_video` bilinearly resizes spatial dims, but `PatchEmbed3D` requires `H % patch_size == 0` (`video_mae.py` line 65). For `patch_size=16` and `image_size=224`, scales like 0.875 (→ 196) or 1.125 (→ 252) yield non-divisible sides and crash the patch embedding. *Multi-scale TTA is therefore not safe with VideoMAE ViT* unless restricted to scales producing patch-divisible sides (e.g. 192/224 ≈ 0.857, 256/224 ≈ 1.143). Rouget was correctly submitted with `training.tta_scales=[1.0]`.
- **Top recommendation**: mirror the Phase-2 150-epoch single-stream best-EMA checkpoint (file path in §A.3, on Thomas's VM under `/Data/thomas.turkieh/.../checkpoints/track_a/`) to this VM every few epochs so the ∼ 42 % recipe survives a host loss this time, let the run finish 150 epochs, and submit with `training.tta_scales=[0.875,1.0,1.`
  and flip-TTA. After that, retrieve and submit *piranha* (highest SSL honest val) with flip-TTA only; rerun *rouget* Phase 2a; complete *truite* Phase 2. Softmax-average the Phase-2 single-stream output with *piranha* for the final submission.

A.2 Inventory by family

Config drift. Notable deviations from YAML / `SSL.md` that remain in the audit trail:

- **SSL-10 roussette pretrain**: resolved `seed`=42 in the log vs. YAML 123 (root `config.yaml` likely overrode the experiment); CLI `pretrain.batch_size`=8 vs YAML 16.
- **SSL-11b rouget finetune trainval**: deliberate CLI `dataset.include_val_in_train=true` resume from the Phase 2a `*.last.pt`; this is the source of the ∼ 60 % leaky val and overwrites `rouget_ft.pt`, so the Phase 2a honest checkpoint is no longer recoverable without a rerun.
- **SSL-12 raie pretrain**: first attempt crashed with `scatter_` dtype mismatch under bf16 AMP (decoder mask token was fp32 while encoder output was bf16); fixed in `src/smth2smth/shared/models/video_mae.py` lines 438–447 by casting `mask_token` to `tokens.dtype` before `scatter_`. Second attempt completed without incident.

Table 6: One row per distinct training job (not per epoch). Honest val means `dataset.use_official_val=true` with no `dataset.include_val_in_train` override.

ID	Family	Preset (`experiment=`)	Status	Honest val (%)	Public (%)
P2-150 (prior)	Single-stream R50+TSM, 150 ep	`track_a_phase2`	DONE (lost ckpt)	–	≈ **42** (lost run)
P2-150 (rerun)	Single-stream R50+TSM, 150 ep	`track_a_phase2`	RUNNING ep38/150	≥39.36 (ema, climbing)	pending
A1	Dual-stream TSM	`track_a_dual_stream_long_class_boost`	STOPPED ep16	31.79 (ema)	–
A1b	Dual-stream TSM (resume)	`...long_class_boost`, `lr`=1.2e-4	DONE	39.47 (ema)	37.69
A2	ViT-B MP from scratch	`track_a_vit_truite`	FAILED	5.41 (ema)	–
A3	ViT-S MP from scratch	`track_a_vit_sardine`	STOPPED ep3	6.15 (live)	–
A4	ViT-B AP from scratch	`track_a_vit_truite_ap`	STOPPED	6.08 (live)	–
–	ViT-B + boosting	`track_a_vit_rouget`	FAILED (NaN)	5.59 (ema)	–
–	ViT-B + balancing	`track_a_vit_roussette`	EARLY STOP	5.23 (ema)	–
–	ViT-B + both	`track_a_vit_raie`	FAILED	6.00 (ema)	–
SSL-9	VideoMAE pretrain (ViT-S)	`track_a_ssl_pretrain_truite`	PRETRAIN ONLY	–	–
SSL-9	VideoMAE FT (ViT-S, 4-h)	`track_a_ssl_finetune_truite`	NOT RUN	–	–
SSL-10	VideoMAE pretrain (ViT-S)	`track_a_ssl_pretrain_roussette`	DONE	–	–
SSL-10	VideoMAE FT (ViT-S, 4-h)	`track_a_ssl_finetune_roussette`	DONE	33.61 (ema, ep27)	–
SSL-11	VideoMAE long MAE (200 ep)	`track_a_ssl_pretrain_rouget`	DONE	–	–
SSL-11a	VideoMAE FT honest 30 ep	`track_a_ssl_finetune_rouget`	DONE	**34.96** (ema, ep30)	–
SSL-11b	VideoMAE FT + val in train	`..._finetune_rouget` + CLI override	STOPPED ep44/60	∼ 60 (leaky)	**36.0**
SSL-12	VideoMAE pretrain (ViT-S)	`track_a_ssl_pretrain_raie`	DONE (retry)	–	–
SSL-12	VideoMAE FT (ViT-S, 6-h)	`track_a_ssl_finetune_raie`	DONE	34.80 (ema, ep29)	–
SSL-13	VideoMAE FT 8-h (planned)	`track_a_ssl_finetune_sole`	NOT RUN	–	–
SSL-14	VideoMAE mask 0.85 (plan)	`track_a_ssl_pretrain_thon`	NOT RUN	–	–
SSL-15	VideoMAE pretrain (ViT-S)	`track_a_ssl_pretrain_piranha`	DONE	–	–
SSL-15	VideoMAE FT 40 ep	`track_a_ssl_finetune_piranha`	DONE	**35.60** (ema, ep32)	–
SSL-16	VideoMAE `lr`=3e-4	`track_a_ssl_pretrain_murene`	NOT RUN (quota)	–	–
DINO	DINO on still frames	`track_a_ssl_pretrain.yaml`	DEPRECATED	inconsistent	–
V-JEPA	V-JEPA clip SSL → CNN	`track_a_vjepa_pretrain/finetune`	LEGACY	–	–

- **Chained launchers**: `chain_raie.sh`, `ssl_rouget_chain.log`, and the piranha pipeline outer log chained pretrain and finetune in a single `nohup`, vs the two-step recipe in `SSL.md` §4. Hydra payloads otherwise match the YAMLs.

A.3 Phase 2 single-stream long-budget rerun — current best path

What the run is. Preset `configs/experiment/track_a_phase2.yaml` on `avanced_resnet50_tsm` (R50 with TSM channel shift inserted in every bottleneck's `conv1`, `shift_div=8`, `shift_place=blockres`), trained *from scratch* (`model.init_from=null`, no SSL warm-start), 4 frames, image size 224, official val. Optimiser AdamW, `lr`=1e-4, `wd`=0.05, cosine schedule with 10-epoch linear warmup and `min_lr`=1e-6 over a 150-epoch budget; early-stopping patience 25. Attentive head with 4 heads, `drop_path_rate`=0.1, `dropout`=0.5, `label_smoothing`=0.1, `videomix_mode=frame_mixup` ($\alpha = 5.0$, $p = 0.5$), AMP fp16, EMA decay 0.999 with both live and EMA evaluated each epoch. RandAugment-T ($n = 2$, $m = 9$, `temporal_plus`) with `random_horizontal_flip=true` and `random_crop` padding 32. At submit time, multi-scale TTA `tta_scales=[0.875, 1.0, 1.125]`, flip with auto left/right pair remap, no logit-adjust.

Status. Run still in progress: at epoch 38/150 the log shows train loss 2.27 (top-1 0.4427), val 0.3629 live and **EMA val** 0.3936 (top-5 0.7420). The previous run with this recipe scored ≈ 42 % public Top-1 on Kaggle but its checkpoint was lost; the live rerun targets the same end-point. Active checkpoint path (on Thomas's host, *not* mirrored on this VM yet):

```
/Data/thomas.turkieh/smth2smth/checkpoints/
track_a/trackA_phase2_randaugT_attn_dropPath0.1_
adamw_ema_amp_4frames_officialVal_ep150_
20260516_best.pt
```

Side-by-side with other best candidates.

Why this recipe wins. Ordered by likely impact.

1. **150-epoch budget with patience 25.** The dual-stream A1b ran the same data stack (frame_mixup, RandAugment-T, official val, AMP, EMA) but early-stopped at epoch 39 with patience 3. Phase 2 keeps the cosine running from 1e-4 down to 1e-6 over a full 150-epoch window, and the EMA accumulates real gain in the late-cosine phase: the log shows monotonic climbs of about +0.6 pp every 3–4 epochs in the 0.30–0.39 band, with no early stall. Truncated at patience 3 this run would have died near ∼ 32 % like A1 did, not ∼ 42 %.

Table 7: Knobs that differ across the four strongest Track-A candidates. Honest val from per-run logs. Public-only column from Kaggle submissions where available.

Knob	Phase 2 R50+TSM 150ep	Dual-stream A1b	SSL piranha	SSL rouget 2b (leaky)
Backbone	R50 + TSM (1-stream)	R50 RGB + R34 diff + fuse	ViT-S (VideoMAE)	ViT-S (VideoMAE long-MAE)
Frames	4	4	4	4
Head	attn, 4h	attn, 4h	attn, 4h	attn, 4h
Optimiser / LR	AdamW / 1e-4	AdamW / 1.2e-4, m_r=0.3	AdamW / 2e-4	AdamW / 2e-4
Weight decay	0.05	0.05	0.05	0.05
Epoch budget	**150**	80 (early-stop ep39)	40 FT (+100 SSL)	30+14 (2a + leaky 2b)
Warmup / min lr	10 / 1e-6	6 / 1e-6	5 / 1e-6	5 / 1e-6
Patience	25	3	15	15
AMP / EMA decay	fp16 / 0.999	bf16 / 0.999	bf16 / 0.9999	bf16 / 0.9999
`drop_path/dropout`	0.1 / 0.5	0.1 / 0.0	0.1 / 0.0	0.1 / 0.0
RandAugment-T	$n = 2$, $m = 9$	$n = 2$, $m = 9$	$n = 4$, $m = 7$	$n = 4$, $m = 7$
`horizontal_flip`	**true**	true	**false**	**false**
videomix	`frame_mixup`, $\alpha = 5$	`frame_mixup`, $\alpha = 5$	`cube_cutmix`, $\alpha = 1$	`cube_cutmix`, $\alpha = 1$
`label_smoothing`	0.1	0.1	0.1	0.1
`class_boosting`	off	**on**	off	off
`class_balance_*`	none / none	none / none	none / none	none / none
`init_from`	**null** (from scratch)	null (silent skip)	piranha_enc	rouget_enc
`tta_scales`	**[0.875, 1.0, 1.125]**	[1.0]	[1.0] (must)	[1.0] (must)
`tta_flip` + L/R remap	on	on	on	on
`tta_logit_adjust`	0.0	0.0	0.0	0.0
Honest val Top-1	$\geq$ 0.3936 (ep 38, climbing)	**0.3947** (ep 39)	0.3560 (ep 32)	$\sim$ 0.60 (leaky)
Public Top-1	$\approx$ 0.42 (lost run)	**0.3769**	–	0.360

2. **CNN + TSM inductive bias at ~ 45 k labelled clips.** Confirmed by §9.12: ViT-B from random init plateaus at $\leq 6\,\%$ val. TSM channel-shift in every bottleneck `conv1` gives temporal context for free with no extra parameters. SSL VideoMAE-ViT recovers part of this (piranha 35.6 % honest val), but at 4 frames and 45 k clips a long-trained CNN still beats it.

3. **Multi-scale TTA `[0.875, 1.0, 1.125]` is safe for this CNN but not for the SSL ViT.** As recorded in §A.4, `PatchEmbed3D` requires $H \bmod 16 = 0$; $0.875 \times 224 = 196$ and $1.125 \times 224 = 252$ are not patch-divisible, so the SSL fish runs must submit with `[1.0]` only. A fully-convolutional R50 has no such constraint, and this asymmetry alone is typically worth $\sim$ 0.5–1.5 pp public Top-1.

4. **Regularisation stack that actually has time to pay off.** `drop_path`=0.1 (linearly scaled) + `dropout`=0.5 + `label_smoothing`=0.1 + frame_mixup α=5.0 + RandAugment-T + EMA 0.999 + AMP all compose coherently. Dual-stream A1b runs the same stack minus `dropout` (it is 0 there) but for only 39 epochs. SSL piranha runs with `dropout`=0 and 40 FT epochs. Phase 2 is the only candidate that pays the regularisation tax *and* trains long enough to claim the credit.

5. **`random_horizontal_flip=true` is safe here because of the submit-time pair remap.** The auto-derived $018 \leftrightarrow 019$ permutation (`src/smth2smth/pipelines/submit.py` lines 213–268) re-aligns the flipped softmax for direction-sensitive classes; for the majority of classes flipping is label-preserving and acts as a free 2× augmentation. The SSL ViT runs play conservative (`horizontal_flip=false`) and forgo this win.

6. **Simpler long-tail handling.** No `class_balance_sampler`, no `class_balance_loss`, no `class_boosting`, no time-reversal, no logit-adjust. Three of the failed supervised ViT runs (rouget / roussette / raie) bolted these on and still plateaued at $\leq 6\,\%$; the dual-stream A1b had `class_boosting` on but it is hard to credit it for the 39.47 % honest val given the very tight early-stop. `frame_mixup` + `label_smoothing` + cosine over 150 epochs gives a softer-but-stronger version of the same long-tail correction without an explicit class weight.

Risks specific to this recipe.

- Checkpoint lives only on Thomas's VM (`/Data/thomas.turkieh/.../*_best.pt`); a single host loss already cost us the $\sim 42\,\%$ once. Mirror the `*_best.pt` to `/Data/romain.poggi/smth2smth/checkpoints/track_a/archive/` every 10 epochs while the rerun is alive.

- The auto pair remap is keyed on class folder names containing `from_left_to_right` / `from_right_to_left`. If the dataset layout drifts, the remap silently becomes identity and `horizontal_flip` starts producing wrong labels. Verify the `[tta] flip-pair remap: ...` line at submit time lists at least one pair.
- `num_classes` in the log is 33 (the legacy on-disk class count); the embedded `trained_class_indices` should be checked against the actual 50-class submission head before the final submit.

A.4 Per-family code verification

Dual-stream RGB+frame-diff TSM. Code: `src/smth2smth/shared/models/dual_stream_rgb_diff_tsm.py`; per-backbone LR groups in `src/smth2smth/pipelines/train.py` (lines 489–497 and the optimiser construction below). **Status: OK, validated end-to-end on Kaggle.** Pitfalls: `model.init_from` is silently skipped for `dual_stream_rgb_diff_tsm` (`train.py` lines 412–416), so attempts to warm-start the dual-stream from any SSL trunk are a no-op without warning at submit time. The exact A1b checkpoint name in the log is not on disk locally; verify `checkpoints/track_a/best_model.pt` before resubmission.

Supervised `video_mae_vit` from scratch. Code: `src/smth2smth/shared/models/video_mae.py` (`VideoMAEViT`, `build_vi` builder routes by `cfg.model.variant` $\in$ {`vit_s`, `vit_b`, `vit_l`} and `cfg.model.head` $\in$ {`mean`, `attn`}. **Status: code is fine; training family is dead at this dataset scale** — see §9.12. Six logs in the audit plateau at $\leq 6\,\%$ val.

VideoMAE SSL pretrain. Code: `src/smth2smth/pipelines/pretrain_videomae.py`; tube mask 0.75 by default, AdamW (0.9, 0.95) wd 0.05, bf16 AMP, cosine LR with linear warmup, encoder-only checkpoint saved every epoch with the `encoder.*` prefix (rewritten in place). **Status: OK after the bf16 `scatter_` fix.** Loss schedule across slots (0.85 $\rightarrow$ 0.25 at 100 ep, 0.234 at 200 ep for *rouget*) is consistent with the reference implementation; `wrote encoder checkpoint -> ... (149 tensors)` on completion.

VideoMAE SSL fine-tune via `model.init_from`. Code: `src/smth2smth/pipelines/train.py` lines 428–441 specialises the load for `video_mae_vit`: the trunk dict already has `encoder.*` keys, so it is loaded `strict=False` *without* the ResNet-specific TSM key renaming (`_ssl_trunk_to_supervised_keys`). All four completed FT logs (*roussette*, *rouget* 2a, *raie*, *piranha*) report "149 encoder tensors, 0 unexpected". **Status: OK.** Pitfall: 40 FT epochs (*piranha*) buys ≤ 0.5 pp over 30 epochs on a cosine schedule that has already annealed; extending the planned *sole*/*thon* FT to 40 ep is not predicted to help materially.

Submit pipeline. Code: `src/smth2smth/pipelines/submit.py`. Three relevant routines: `_build_untrained_mask` ([line 186]) sets logits at never-trained class indices to $-\infty$; `_build_flip_class_permutation` ([line 213]) auto-derives the 018/019 left/right pair remap; `_rescale_video` ([line 292]) bilinearly resizes spatial dims for multi-scale TTA. **Status: OK with one VideoMAE-specific restriction.** Pitfalls:

1. **Multi-scale TTA + VideoMAE**: each scaled clip must have H, W divisible by `model.patch_size`=16. At `image_size`=224 the default $\{0.875, 1.0, 1.125\}$ produces $\{196, 224, 252\}$; 196 and 252 are not patch-divisible and would crash `PatchEmbed3D`. Submit VideoMAE ViT runs with `training.tta_scales=[1.0]` (already done for *rouget*).
2. **Untrained-class mask**: derived from `trained_class_indices` embedded in the checkpoint. When the head width or class folder set is updated, regenerate the mask by retraining or by clearing the stale list, otherwise valid logits get silently zeroed.
3. **`model.pretrained` flag**: `submit.py` reads `saved_cfg.model.pretrained` to decide on ImageNet normalisation; VideoMAE runs save `pretrained: false` (no normalisation), so no action needed for the current submissions.

Legacy DINO / V-JEPA. Code retained for the report (§4.1, §4); `_ssl_trunk_to_supervised_keys` continues to handle both flavours. **Status: deprecated for new runs.** The DINO ablation inconsistency and the V-JEPA $\rightarrow$ CNN path are both superseded by dual-stream + VideoMAE.

Table 8: Per-family action. Justification ties to competition rules in `docs/track_a_desc.txt` (closed world, top-1 evaluation) and on-disk honest val.

Family	Verdict	Justification
Phase 2 R50+TSM 150 ep	**KEEP — primary path**	Prior run reached ≈ 42 % public Top-1; rerun in progress under `logs/trackA_phase2_..._ep150_20260516.log`, ep 38/150 at EMA val 0.3936 and climbing. Mirror the `*_best.pt` regularly.
Dual-stream TSM (A1/A1b)	**KEEP**	Verified 37.69 % public; second-best single-model submission. Useful for ensembling with Phase 2 (independent error modes: single-stream vs. RGB+motion).
Supervised ViT from scratch	**DISCARD**	6 distinct logs converge at ≤ 6 % val (chance scaled). Verdict already in §9.12.
SSL-15 piranha	**KEEP**	Highest honest val (35.60 %); 40-FT-epoch budget paid. Action: retrieve weights and submit.
SSL-11 rouget (2a)	**TWEAK**	34.96 % honest val proved the recipe; current `rouget_ft.pt` is the leaky Phase 2b copy. Rerun Phase 2a from `rouget_encoder.pt` (on disk).
SSL-12 raie	**KEEP**	34.80 % honest val with 6-head probe; submit when checkpoint is retrievable.
SSL-10 roussette	**KEEP/TWEAK**	33.61 % honest val with seed/bs drift. Acceptable if cheap; prefer piranha / raie / rouget if budget is tight.
SSL-9 truite	**TWEAK**	Pretrain on disk; FT never launched. Cheap completion via `experiment=track_a_ssl_finetune_truite`.
SSL-13/14/16 (sole, thon, murene)	**DEFER**	Each is a fresh ∼ 10 h pretrain+FT cycle; with 35.60 % already in hand, only run if ablating `mask_ratio` (thon) or probe width (sole) is the explicit goal.
SSL-8 barbeau	**DEFER**	Thomas' slot; not run.
Legacy DINO	**DISCARD** for new runs	§4.1; retained as a report baseline.
Legacy V-JEPA	**DEFER**	Superseded by dual-stream + VideoMAE; revisit only if both plateau.

A.5 Recommendations

Ranked top-3 actionable next runs.

1. **Protect the Phase 2 150-epoch checkpoint and submit it.** Mirror `trackA_phase2_..._ep150_20260516_best.pt` from `/Data/thomas.turkieh/.../checkpoints/track_a/` (full path in §A.3) to `checkpoints/track_a/archive/` every ∼ 10 epochs while the rerun is alive. When the run finishes (or its EMA val plateaus near 0.42):

```
PYTHONPATH=src uv run python -m smth2smth.pipelines.submit \
    experiment=track_a_phase2 \
    training.checkpoint_path=<path-to-mirrored-best.pt> \
    training.tta=true training.tta_flip=true \
    training.tta_scales=[0.875,1.0,1.125] \
    dataset.submission_output=submissions/track_a_phase2_150ep.csv
```

   Expected public Top-1: ∼ 42 % (target of the prior lost run).

2. **Recover and submit piranha** (independent ViT path, ensemble candidate). After copying `piranha_encoder.pt` and `piranha_ft.pt` into `checkpoints/track_a/ssl/`:

```
PYTHONPATH=src uv run python -m smth2smth.pipelines.submit \
    experiment=track_a_ssl_finetune_piranha \
    training.checkpoint_path=checkpoints/track_a/ssl/piranha_ft.pt \
    training.tta=true training.tta_flip=true training.tta_scales=[1.0] \
    training.tta_logit_adjust=1.0 \
    dataset.submission_output=submissions/track_a_piranha_ssl.csv
```

   Expected public Top-1: 36–38 % (rouget's 34.96 % → 36.0 % honest-to-public correspondence applied to piranha's higher honest val).

3. **Rerun rouget Phase 2a cleanly** and resubmit. The on-disk `rouget_ft.pt` is the leaky Phase 2b weights; the honest checkpoint was overwritten. Use the existing encoder:

```
PYTHONPATH=src uv run python -m smth2smth.pipelines.train \
    experiment=track_a_ssl_finetune_rouget \
    model.init_from=checkpoints/track_a/ssl/rouget_encoder.pt
# Then submit with tta_scales=[1.0], tta_flip=true.
```

Cost: ∼ 3 h. Re-grounds rouget at 34.96 % honest; archive the existing leaky `rouget_ft.pt` first.

4. **Finish truite Phase 2** (encoder already on disk):

```
PYTHONPATH=src uv run python -m smth2smth.pipelines.train \
    experiment=track_a_ssl_finetune_truite
# model.init_from defaults to truite_encoder.pt in the YAML.
```

Cost: ∼ 2.5 h FT. Expected: ∼ 34 % honest val based on sibling runs. Gives a fourth comparable VideoMAE SSL data point for ensembling.

Ensembling note. After items 1–3, softmax-averaging the Phase 2 single-stream R50+TSM (item 1) with piranha (item 2, ViT-S) at submit time is the most leverage available without a new training run, because the two trunks have independent error modes (CNN-temporal-shift vs. transformer-tube-attention). Apply flip-TTA + left/right remap per model before averaging. Dual-stream A1b can be folded in as a third member if its checkpoint identity is confirmed.

A.6 Open risks and missing logs

- **Phase 2 150-ep rerun checkpoint is on a foreign VM.** The live target is `trackA_phase2_..._ep150_20260516_best.p` under `/Data/thomas.turkieh/.../checkpoints/track_a/`; full path in §A.3. This is the single point of failure for reproducing the ∼ 42 % result. Mirror to `checkpoints/track_a/archive/` on a schedule; do not rely on the producing host surviving alone.
- **Missing local SSL checkpoints.** Only `rouget_encoder.pt` and the leaky `rouget_ft.pt` are present under `checkpoints/track_a/ssl/`. All references to `piranha_*.pt`, `raie_*.pt`, `roussette_*.pt` and `truite_encoder.pt` in `experiments.md` and the run log (§9.11) point at files that live on the producing VMs and must be transferred before submission.
- **Dual-stream A1b checkpoint identity.** The submit log names `dual_stream_long_class_boost_lr1_20260515_025557` which is not present locally. `checkpoints/track_a/best_model.pt` (∼ 1 GiB, 15 May 23:37) is consistent in size with the dual-stream model but its identity must be confirmed (e.g. via the embedded `checkpoint_kind`, `val_top1` in `extra`) before any new submission relies on it.
- **Stale `trained_class_indices`.** Some early checkpoints recorded 33 trained classes (the old folder layout). If a VideoMAE submission is generated from such a checkpoint while the head is 50-wide, the un-trained mask silently zeroes valid logits. Sanity-check `len(checkpoint["extra"]["trained_class_indices"])` matches `num_classes` before each submit.
- **Phase 2b rouget is the only Kaggle SSL signal so far.** The 36.0 % public score was produced by the leaky checkpoint; we lack a direct public number for any cleanly-trained SSL FT. Recommendation #1 (piranha) and #2 (rouget Phase 2a) close this gap.
- **`submissions/track_a.csv` provenance unclear.** Dated 16 May 14:04, same byte count as `submissions/track_a_rouget_` likely a duplicate but should be verified before referring to it on the leaderboard.
- **Thomas slots 1–8** (`SSL.md`) have no logs on this VM, so the audit can only confirm "not run here". If those checkpoints exist on Thomas' machines, fold them into the inventory before the next sweep decision.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* ICCV 2015.

[2] G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Q. Weinberger. *Deep Networks with Stochastic Depth.* ECCV 2016.

[3] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. *Emerging Properties in Self-Supervised Vision Transformers.* ICCV 2021.

[4] B. T. Polyak, A. B. Juditsky. *Acceleration of Stochastic Approximation by Averaging.* SIAM J. Control and Optimization, 1992.

[5] J. Lin, C. Gan, S. Han. *TSM: Temporal Shift Module for Efficient Video Understanding.* ICCV 2019.

[6] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization.* ICLR 2019.

[7] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, N. Ballas. *Revisiting Feature Prediction for Learning Visual Representations from Video* (V-JEPA). TMLR 2024.

[8] M. Assran et al. *V-JEPA 2: Scaling Joint-Embedding Predictive Architectures for Self-Supervised Video Representation Learning.* arXiv:2506.09985, 2025.

[9] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, S. Belongie. *Class-Balanced Loss Based on Effective Number of Samples.* CVPR 2019.

[10] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, S. Kumar. *Long-Tail Learning via Logit Adjustment.* ICLR 2021.